

SVEUČILIŠTE JURJA DOBRILE U PULI
FAKULTET INFORMATIKE

David Kokalović

**RAZVOJ MOBILNE APLIKACIJE ZA PREPOZNAVANJE
GRAĐEVINSKIH PROBLEMA PRIMJENOM UMJETNE
INTELIGENCIJE**

ZAVRŠNI RAD

Pula, 2026.

SVEUČILIŠTE JURJA DOBRILE U PULI
FAKULTET INFORMATIKE

David Kokalović

**RAZVOJ MOBILNE APLIKACIJE ZA PREPOZNAVANJE
GRAĐEVINSKIH PROBLEMA PRIMJENOM UMJETNE
INTELIGENCIJE**

ZAVRŠNI RAD

JMBAG: 0303107334

Status: Redoviti student

Studijski smjer: Informatika

Kolegij: Web aplikacije

Znanstveno područje: Društvene znanosti

Mentor: izv. prof. dr. sc. Nikola Tanković

Pula, 2026.

Sažetak

Ovaj završni rad bavi se izradom web aplikacije za automatiziranu dijagnostiku građevinskih problema, uz pomoć umjetne inteligencije. Na temelju fotografija i tekstualnog opisa problema, koje unosi korisnik, provedeni su ključni koraci razvoja: analiza zahtjeva i dizajn arhitekture sustava, implementacija REST API-ja i modela podataka integracija s vanjskim AI servisom za analizu fotografije, izrada korisničkog sučelja za prijavu problema i pregled rezultata analize.

Backend sustav izrađen je korištenjem Node.js okruženja i Express.js frameworka, uz MongoDB Atlas kao NoSQL bazu podataka kojoj se pristupa putem Mongoose ODM-a. Autentikacija korisnika ostvarena je JWT tokenima uz hashiranje lozinke algoritmom bcrypt, a validacija ulaznih podataka provedena je na razini API-ja korištenjem express-validator biblioteke. Frontend sučelje razvijeno je u Vue.js 3 frameworku primjenom Composition API-ja, s komunikacijom prema backendu putem Axios HTTP klijenta. Za automatsku analizu fotografija i generiranje dijagnoze problema korišten je Anthropic Claude API, čiji rezultat (kategorija problema, procjena hitnosti i troškova) sustav dalje koristi za automatsko povezivanje korisnika s odgovarajućim lokalnim obrtnicima iz baze podataka.

Rad pokazuje kako se kombinacijom modernih web tehnologija i vanjskih AI servisa mogu automatizirati zadaci koji su tradicionalno zahtijevali stručnu procjenu na terenu, čime se korisnicima omogućuje brža i pristupačnija dijagnostika građevinskih problema.

Ključne riječi: web aplikacija, umjetna inteligencija, Node.js, Express.js, Vue.js, MongoDB, REST API, JWT autentikacija

Abstract

This thesis deals with the development of a web application for automated diagnosis of construction-related problems, using artificial intelligence. Based on a photograph and text description of the problem, provided by the user, the key development steps were carried out: requirements analysis and system architecture design, implementation of the REST API and data model, integration with an external AI service for photo analysis, and development of a user interface for reporting problems and reviewing analysis results.

The backend was built using the Node.js runtime and the Express.js, framework, with MongoDB Atlas as a NoSQL database accessed through the Mongoose ODM. User authentication was implemented using JWT tokens with password hashing via bcrypt, and input validation was enforced at the API level using the express-validator library. The frontend interface was developed in the Vue.js 3 framework using the Composition API, communicating with the backend via the Axios HTTP client. The Anthropic Claude API was used for automatic photo analysis and diagnosis generation, whose output (problem category, urgency and cost estimate) the system then uses to automatically connect the user with suitable

local contractors from the database.

The thesis demonstrates how combining modern web technologies with external AI services can automate tasks that traditionally required expert on-site assessment, providing users with faster and more accessible diagnosis of construction problems.

Keywords: web application, artificial intelligence, Node.js, Express.js, Vue.js, MongoDB, REST API, JWT authentication

Sadržaj

1	Uvod	1
2	Pregled korištenih tehnologija	2
2.1	Tehnologije za korisnike	2
2.2	Tehnologije za poslužitelje	2
2.3	Baza podataka	3
2.4	Autentikacija i sigurnost	3
2.5	Umjetna inteligencija – Anthropic Claude API	4
3	Analiza i dizajn sustava	5
3.1	Opis problema i ciljevi aplikacije	5
3.2	Funkcionalni zahtjevi	5
3.3	Nefunkcionalni zahtjevi	6
3.4	Arhitektura sustava	7
3.5	Akteri i slučajevi korištenja	7
3.6	Konceptualni model	8
3.7	Implementacija sheme u MongoDB-u	9
3.8	Dizajn REST API-ja	12
4	Implementacija	14
4.1	Implementacija poslužiteljskog dijela	14
4.2	Implementacija korisničkog dijela	23
4.3	Sigurnosni aspekti implementacije	32
4.4	Proširenja sustava	32
5	Testiranje	43
6	Zaključak	45
	Literatura	46
	Popis slika	47
	Popis tablica	48

1 Uvod

Kada se u domaćinstvu, nekom vanjskom objektu ili poslovnom prostoru pojavi građevinski problem (vlažna mrlja na zidu, pukotina, oštećenje krova ili kvar na vodoinstalacijama) prosječan korisnik najčešće ne zna procijeniti o kakvom se problemu radi, koliko je on hitan, koliki je okvirni trošak sanacije, niti kojeg točno obrtnika treba kontaktirati. Posljedica toga je da se ili posao odgađa dok problem ne postane ozbiljniji i skuplji, ili se angažira pogrešan izvođač, ili se plaća nepotreban izlazak na teren samo radi procjene stanja. U okviru ovog rada razvijena je web aplikacija čiji je cilj rješavanje upravo tog problema: korisnik fotografira uočeni problem, opisuje ga vlastitim riječima, a aplikacija uz pomoć umjetne inteligencije generira dijagnozu, procjenu hitnosti, okvirni raspon troška sanacije i preporuku lokalnih obrtnika koji se bave odgovarajućom kategorijom problema. Cilj ovog završnog rada je opisati proces razvoja te aplikacije od definiranja problema, odabira tehnologija, preko dizajna arhitekture sustava, do same implementacije backend i frontend dijela aplikacije i sigurnosnih mehanizama koji štite podatke korisnika. Aplikacija je podijeljena na dva neovisna dijela koji komuniciraju isključivo putem REST API-ja: backend (izgrađen u Node.js okruženju uz Express.js, framework i MongoDB bazu podataka) koji upravlja poslovnom logikom, autentikacijom i komunikacijom s vanjskim AI servisom, te frontend (izgrađen u Vue.js 3 frameworku) koji korisniku pruža sučelje za prijavu problema, pregled rezultata analize i uvid u povijest prijava. Za automatsku analizu fotografija korišten je Anthropic Claude API, koji na temelju poslane slike i tekstualnog opisa problema vraća strukturiran odgovor naslov problema, kategoriju, opis, preporučeno rješenje, procjenu hitnosti i troška. Na temelju prepoznate kategorije, backend iz baze podataka dohvaća obrtnike registrirane za tu kategoriju rada te ih predlaže korisniku, čime se zatvara cijeli tijek od uočenog problema do konkretnog kontakta koji ga može riješiti. Rad je strukturiran na sljedeći način: Poglavlje 2 daje pregled tehnologija i alata korištenih u razvoju aplikacije, uz obrazloženje zašto su upravo one odabrane. Poglavlje 3 opisuje analizu problema i dizajn sustava, uključujući arhitekturu, konceptualni model podataka i dizajn REST API-ja. Poglavlje 4 detaljno opisuje implementaciju backend i frontend dijela aplikacije, uključujući autentikaciju, obradu fotografija i komunikaciju s AI servisom. Poglavlje 5 opisuje način na koji je aplikacija testirana. Poglavlje 6 donosi zaključak rada, s osvrtom na ostvarene ciljeve i moguća buduća proširenja aplikacije.

2 Pregled korištenih tehnologija

U ovom poglavlju opisane su tehnologije i alati korišteni u razvoju aplikacije, uz obrazloženje razloga njihova odabira u odnosu na alternativna rješenja.

2.1 Tehnologije za korisnike

Za izradu korisničkog sučelja korišten je **Vue.js 3** [1], progresivni JavaScript framework za izgradnju interaktivnih web aplikacija. Vue.js omogućuje deklarativan pristup izradi sučelja – programer opisuje *što* se treba prikazati, dok framework interno upravlja ažuriranjem stvarnog DOM-a, kad se promijene podaci. Aplikacija koristi **Composition API** (kroz sintaksu `<script setup>`), noviji način organizacije Vue komponenti u kojem se kod grupira prema logičkoj cjelini funkcionalnosti, a ne prema vrsti (podaci, metode, računate vrijednosti), što olakšava čitljivost i ponovno korištenje koda, kroz takozvane composable funkcije.

Reaktivnost podataka ostvarena je funkcijama `ref()` (za primitivne vrijednosti) i `reactive()` (za objekte), dok se izvedene vrijednosti, koje ovise o drugim reaktivnim podacima definiraju s pomoću `computed()`. Za pokretanje razvojnog okruženja i izradu redukcijske verzije aplikacije korišten je **Vite** [2], alat koji u odnosu na starije bundler (npr. Webpack) nudi znatno brže pokretanje razvojnog servera i takozvani Hot Module Replacement – trenutni prikaz promjena u kodu bez potrebe za ručnim osvježavanjem stranice.

Navigacija između različitih dijelova aplikacije (prijava, glavna stranica, administratorski panel) implementirana je pomoću **Vue Router** [3] biblioteke, koja mapira URL putanje na odgovarajuće komponente i omogućuje tzv. *client-side routing* – promjenu prikazane komponente bez ponovnog zahtjeva cijele stranice prema serveru, čime aplikacija funkcionira kao *Single Page Application* (SPA).

Za komunikaciju s backend serverom korištena je **Axios** [4] biblioteka, umjesto ugrađenog `fetch` API-ja, zbog jednostavnijeg rukovanja greškama, mogućnosti kreiranja konfigurirane instance s unaprijed postavljenom baznom adresom (`baseURL`), te podrške za tzv. interceptore – funkcije koje se automatski izvršavaju za svaki odlazni zahtjev ili dolazni odgovor. Ova mogućnost iskorištena je za automatsko dodavanje JWT autentikacijskog tokena u zaglavlje svakog zahtjeva prema zaštićenim rutama, bez potrebe da se to ručno ponavlja za svaku komponentu zasebno.

2.2 Tehnologije za poslužitelje

Poslužiteljska strana aplikacije izgrađena je na **Node.js**, izvršnom okruženju koje omogućuje pokretanje JavaScript koda izvan preglednika, korištenje istog programskog jezika na frontendu i backendu. Nad Node.js okruženjem korišten je **Express.js** [5], minimalistički web framework koji pojednostavljuje definiranje ruta (kombinacija HTTP metode i putanje), obradu zahtjeva i odgovora, te organizaciju koda kroz koncept *middleware* funkcija.

Middleware funkcije u Express.js-u izvršavaju se između primitka zahtjeva i slanja konačnog odgovora – mogu modificirati zahtjev, provjeriti uvjete (npr. va-

ljanost autentikacijskog tokena) i pozivom funkcije `next()` proslijediti kontrolu sljedećoj funkciji u lancu. Ovaj koncept iskorišten je za organizaciju više neovisnih odgovornosti: provjeru CORS pravila, parsiranje JSON tijela zahtjeva, provjeru autentikacije, obradu uploada datoteka te centralizirano rukovanje greškama.

Rute aplikacije organizirane su kroz `express.Router()` instance, grupirane prema resursu nad kojim djeluju (korisnici, analize, obrtnici, izraženi interesi, administracija), pri čemu svaka grupa ruta predstavlja zaseban modul montiran, na glavnu aplikaciju s vlastitim prefiksom putanje. Ovakva organizacija koda slijedi REST arhitekturni stil, opisan detaljnije u poglavlju 3.

2.3 Baza podataka

Za pohranu podataka korišten je **MongoDB** [6, 7], NoSQL baza podataka koja podatke sprema kao dokumente u BSON formatu (binarna inačica JSON-a) grupirane u kolekcije. Za razliku od relacijskih baza podataka, MongoDB ne zahtijeva strogo definiranu shemu na razini baze, te omogućuje prirodno spremanje ugniježđenih struktura (npr. polje kategorija unutar dokumenta obrtnika), bez potrebe za dodatnim tablicama i JOIN operacijama. Ovakav pristup odgovara prirodi podataka u aplikaciji, gdje su odnosi između entiteta jednostavni a dio podataka je prirodno ugniježđen.

Baza je hostirana na **MongoDB Atlas** platformi, službenoj cloud usluzi za MongoDB, koja u besplatnoj inačici (M0 tier), pruža dovoljno resursa za razvoj i demonstraciju aplikacije ovog opsega. Atlas, automatski upravlja sigurnosnim kopijama i mrežnim pristupom baze putem IP whitelistinga, te je dostupan s bilo kojeg računala bez potrebe za lokalnom instalacijom poslužitelja baze podataka.

Za komunikaciju s bazom iz Node.js aplikacije korišten je **Mongoose** [8], ODM (Object Document Mapper) biblioteka koja nad MongoDB dokumentima nameće shemu na razini aplikacije – definira tipove polja, obavezna polja, dopuštene vrijednosti (enum) i validacijska pravila prije spremanja podataka u bazu. Mongoose modeli korišteni u aplikaciji opisani su detaljnije u poglavlju 3, u sklopu modela podataka.

2.4 Autentikacija i sigurnost

Autentikacija korisnika, ostvarena je pomoću **JWT** (JSON Web Token) tokena [9]. JWT je string podijeljen u tri dijela - zaglavlje, podatkovni dio (payload) i digitalni potpis - pri čemu potpis, izračunat pomoću tajnog ključa poznatog samo serveru, jamči da sadržaj tokena nije naknadno izmijenjen. Ovakav pristup omogućuje autentikaciju, bez potrebe da server pamti stanje sesije, za svakog korisnika (tzv. *stateless* pristup), što je u skladu s REST arhitekturnim principima opisanim u sljedećem poglavlju: svaki zahtjev prema zaštićenoj ruti mora samostalno nositi token u **Authorization** zaglavlju, koji, middleware funkcija provjerava prije nego dopusti pristup resursu.

Lozinke korisnika prije spremanja u bazu podataka hashiraju se algoritmom **bcrypt** [10], koji, uključuje tzv. *salt* – nasumični niz dodan lozinki prije hashiranja – čime se onemogućuje korištenje unaprijed izračunatih tablica (rainbow tables),

za probijanje lozinki. Pri prijavi korisnika, unesena lozinka se ne dešifrira, već se ponovno hashira i uspoređuje s pohranjenim hashom.

Za zaštitu od neovlaštenih zahtjeva s drugih domena konfiguriran je **CORS** (Cross-Origin Resource Sharing) mehanizam, koji eksplicitno definira s kojih je adresa (origin) dopušteno pozivati API. Dodatno, ulazni podaci svih zahtjeva validiraju se na razini API-ja pomoću biblioteke **express-validator**, čime se sprječava spremanje neispravnih ili zlonamjernih podataka u bazu – validacija na frontendu (npr. HTML5 atributi obrasca) služi isključivo za brzu povratnu informaciju korisniku i nije dovoljna sama za sebe, jer se može zaobići izravnim pozivom prema API-ju.

2.5 Umjetna inteligencija – Anthropic Claude API

Za automatsku analizu fotografija građevinskih problema korišten je **Anthropic Claude API** [11], odnosno njegova mogućnost analize slikovnog sadržaja (tzv. *vision* funkcionalnost). Fotografija koju korisnik učitava, zajedno s tekstualnim opisom problema, šalje se modelu u obliku base64 kodiranog sadržaja unutar zahtjeva prema API-ju. Model zatim vraća strukturiran tekstualni odgovor koji backend parsira u konkretna polja – naslov problema, kategoriju, opis, preporučeno rješenje, procjenu hitnosti te okvirni raspon troška sanacije.

Odabir ovakvog pristupa, umjesto treniranja vlastitog modela strojnog učenja, omogućio je fokus na razvoj same aplikacije i njezine poslovne logike, dok je zadatak prepoznavanja i klasifikacije problema na fotografiji prepušten unaprijed treniranom, općenito sposobnom modelu dostupnom putem API-ja.

3 Analiza i dizajn sustava

U ovom ćemo poglavlju analizirati se problem koji aplikacija rješava te se definiraju funkcionalni i nefunkcionalni zahtjevi sustava. Opisuje se arhitektura sustava, akteri i slučajevi korištenja, konceptualni model podataka i njegova implementacija u MongoDB bazi, zaključno s dizajnom REST API-ja koji povezuje frontend i backend dio aplikacije.

3.1 Opis problema i ciljevi aplikacije

Cilj razvijene aplikacije je korisnicima omogućiti brzu i pristupačnu dijagnostiku uočenih građevinskih problema, bez potrebe za inicijalnim angažiranjem stručnjaka, radi same procjene stanja. Osnovni tijek korištenja aplikacije sastoji se od sljedećih koraka: korisnik se registrira i prijavljuje u sustav, fotografira uočeni problem i unosi kratak tekstualni opis, sustav putem vanjskog AI servisa analizira fotografiju i opis te generira strukturiranu dijagnozu, nakon čega sustav korisniku predlaže lokalne obrtnike registrirane za odgovarajuću kategoriju problema. Korisnik, zatim može izraziti interes za kontakt s predloženim obrtnikom, čime se cijeli tijek od uočenog problema do konkretnog kontakta zatvara unutar jedne aplikacije.

3.2 Funkcionalni zahtjevi

Na temelju opisanog cilja definirani su sljedeći funkcionalni zahtjevi sustava:

- registracija i prijava korisnika uz autentikaciju putem JWT tokena,
- upload fotografije uočenog problema uz tekstualni opis,
- automatska AI analiza fotografije i generiranje dijagnoze (kategorija, hitnost, procjena troška, preporučeno rješenje),
- prikaz obrtnika registriranih za prepoznatu kategoriju problema,
- mogućnost izražavanja interesa korisnika za kontakt s obrtnikom,
- pregled povijesti prethodnih analiza pojedinog korisnika,
- administratorski uvid u popis korisnika, analiza, obrtnika i izraženih interesa, uz mogućnost upravljanja obrtnicima.
- pretraga obrtnika po nazivu ili gradu u administratorskom sučelju,
- promjena statusa izraženog interesa (na čekanju, kontaktirano, završeno),
- brisanje vlastite analize od strane korisnika, uz automatsko brisanje povezanih izraženih interesa,
- administratorski dashboard s objedinjenom statistikom sustava (brojkorisnika, analiza po kategoriji, interesa po statusu, prosječna ocjena obrtnika),

- ocjenjivanje obrtnika od strane korisnika nakon izraženog interesa,
- uređivanje vlastitog profila i zasebna promjena lozinke uz provjeru trenutne lozinke,
- samostalna registracija obrtnika, bez potrebe za posredovanjem administratora,
- filtriranje popisa analiza po kategoriji problema i vremenskom razdoblju,
- sortiranje i izvoz podataka (korisnici, analize, interesi, obrtnici) u CSV format.

3.3 Nefunkcionalni zahtjevi

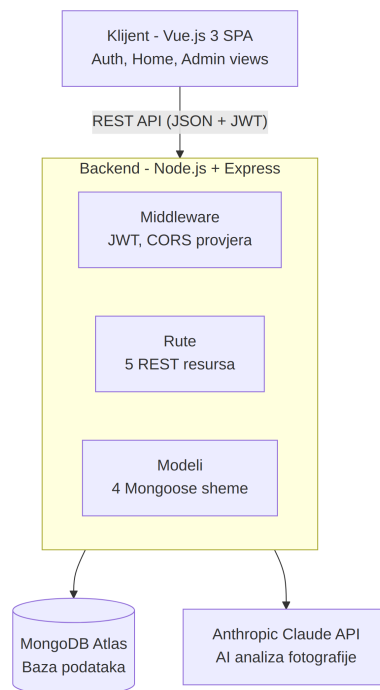
Osim funkcionalnih zahtjeva opisanih u prethodnom potpoglavlju, pri dizajnu sustava uzeti su u obzir i sljedeći nefunkcionalni zahtjevi:

- **Sigurnost** – svi osjetljivi podaci (lozinke, JWT tajni ključ, API ključevi) moraju biti zaštićeni, nikad izravno dostupni korisničkoj aplikaciji; komunikacija s vanjskim servisima (MongoDB Atlas, Anthropic Claude API) odvija se isključivo sa strane servera.
- **Autorizacija na razini resursa** – korisnik smije pristupiti isključivo vlastitim podacima (analize, fotografije, izraženi interesi), osim u slučaju administratorske uloge koja ima uvid u sve zapise.
- **Odzivnost sučelja** – korisničko sučelje mora ostati responzivno tijekom asinkronih operacija (npr. čekanje odgovora AI servisa), uz jasnu vizualnu naznaku stanja učitavanja.
- **Prenosivost** – razdvojenost frontend i backend dijela putem REST API-ja omogućuje neovisan deployment svakog dijela na različitim uslugama (npr. Vercel za frontend, Render za backend) bez izmjena u kodu.
- **Robusnost na pogreške vanjskih servisa** – sustav mora smisleno obraditi situacije u kojima AI servis vrati nevaljan ili neočekivan odgovor (npr. odgovor koji ne odgovara očekivanom JSON formatu), umjesto da takva greška sruši server.
- **Skalabilnost baze podataka** – korištenje dokumentne NoSQL baze (MongoDB) omogućuje jednostavno horizontalno skaliranje i fleksibilnu evoluciju sheme podataka s rastom broja korisnika i analiza.

Navedeni zahtjevi izravno su utjecali na arhitekturne odluke opisane u nastavku poglavlja, posebice na odabir stateless JWT autentikacije, centraliziranog rukovanja greškama (potpoglavlje 4.1.4) i strogog razdvajanja frontend i backend odgovornosti (potpoglavlje 3.4).

3.4 Arhitektura sustava

Sustav prepoznaje tri aktera s različitim razinama ovlasti: korisnik (client) – korisnik koji prijavljuje građevinski problem i traži dijagnozu, obrtnik (contractor) – izvođač radova koji se putem sustava povezuje s korisnicima, a svoj profil može registrirati i samostalno, bez posredovanja administratora, te administrator (admin) – korisnik s punim uvidom i upravljanjem svim podacima sustava. Ovakvo razdvajanje omogućuje neovisan razvoj, testiranje i eventualni deployment svakog dijela, te osigurava da svi osjetljivi podaci (pristupni ključevi, tajni ključ za potpisivanje JWT tokena) ostanu isključivo na strani servera, izvan dosega korisnika. Backend je jedini dio sustava koji izravno komunicira s bazom podataka i vanjskim AI servisom. Arhitektura sustava prikazana je na slici 1.



Slika 1: Arhitektura sustava

Backend je interno organiziran kroz tri sloja: middleware sloj koji provodi provjeru autentikacije i CORS pravila, sloj ruta koji definira dostupne REST resurse, te sloj modela koji putem Mongoose ODM-a definira strukturu podataka i komunicira s MongoDB Atlas bazom. Fotografije koje korisnik učitava analiziraju se slanjem prema Anthropic Claude API-ju, čiji odgovor backend obrađuje i sprema u bazu podataka.

3.5 Akteri i slučajevi korištenja

Sustav prepoznaje tri aktera s različitim razinama ovlasti: **korisnik** (client) – korisnik koji prijavljuje građevinski problem i traži dijagnozu, **obrtnik** (contractor) – registrirani izvođač radova koji se putem sustava povezuje s korisnicima, te **ad-**

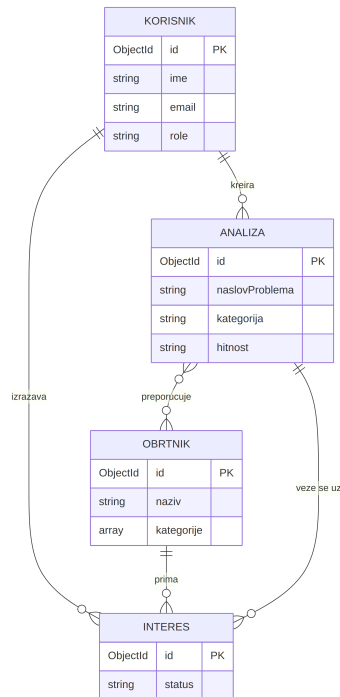
administrator (admin) – korisnik s punim uvidom i upravljanjem svim podacima sustava. Ključni slučajevi korištenja za svakog aktera sažeti su u tablici 1.

Akter	Slučaj korištenja	Opis
Korisnik	Registracija/prijava	Kreiranje računa i autentikacija putem JWT tokena
Korisnik	Prijava problema	Upload fotografije uz tekstualni opis uočenog problema
Korisnik	Pregled dijagnoze	Uvid u AI generiranu kategoriju, hitnost i procjenu troška
Korisnik	Izražavanje interesa	Kontaktiranje predloženog obrtnika za konkretnu analizu
Korisnik	Pregled povijesti	Uvid u sve prethodno podnesene analize
Obrtnik	Pregled zaprimljenih interesa	Uvid u korisnike koji su izrazili interes za kontakt
Administrator	Upravljanje obrtnicima	Dodavanje, uređivanje i brisanje zapisa obrtnika
Administrator	Nadzor sustava	Uvid u sve korisnike, analize i izražene interese
Korisnik	Ocjenjivanje obrtnika	Ocjena i komentar nakon izraženog interesa za obrtnika
Korisnik	Uređivanje profila	Izmjena osobnih podataka i promjena lozinke
Obrtnik	Samostalna registracija	Kreiranje vlastitog profila obrta prilikom registracije
Administrator	Pregled statistike	Uvid u objedinjene brojčane pokazatelje sustava

Tablica 1: Pregled aktera i slučajeva korištenja sustava

3.6 Konceptualni model

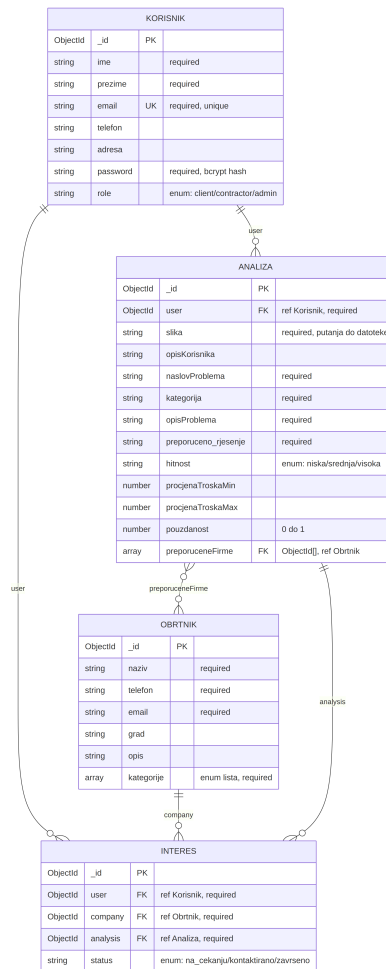
Konceptualni model obuhvaća četiri ključna entiteta: korisnika (**User**), obrtnika (**Company**), analize (**Analysis**) i izraženog interesa (**Interest**). Entitet **Analysis** predstavlja rezultat AI dijagnoze nad fotografijom koju je korisnik učitao, dok **Interest** povezuje korisnika, obrtnika i konkretnu analizu u trenutku kad korisnik izrazi interes za kontakt. Kardinalnosti su čitljive: jedan korisnik može imati više analiza i izraženih interesa, jedan obrtnik može biti primatelj više izraženih interesa, a jedna analiza može preporučiti više obrtnika istovremeno (i obrnuto), što je veza tipa više-na-više. Konceptualni model prikazan je na slici 2.



Slika 2: Konceptualni model podataka

3.7 Implementacija sheme u MongoDB-u

Za razliku od relacijskih baza podataka, gdje se konceptualni model nakon izrade dodatno transformira u zaseban relacijski model normaliziran prema pravilima normalnih formi, MongoDB kao dokumentna NoSQL baza ne zahtijeva taj dodatni korak – dokumenti mogu izravno sadržavati ugniježdene strukture (npr. polje niza kategorija unutar dokumenta obrtnika) bez potrebe za rastavljanjem u zasebne tablice i JOIN operacijama. Konceptualni model stoga se u ovom slučaju izravno implementira kroz Mongoose sheme, koje na razini aplikacije nameću strukturu, tipove podataka i validacijska pravila nad inače fleksibilnim MongoDB dokumentima. Detaljna shema, s tipovima polja i vezama između kolekcija, prikazana je na slici 3.



Slika 3: Implementacija Mongoose shema

Stvarna implementacija četiriju Mongoose shema, s tipovima polja, obaveznim vrijednostima i validacijskim pravilima, prikazana je u nastavku.

```

1  const userSchema = new mongoose.Schema(
2    {
3      ime: { type: String, required: true, trim: true },
4      prezime: { type: String, required: true, trim: true },
5      email: { type: String, required: true, unique: true, lowercase
6        : true, trim: true },
7      telefon: { type: String, trim: true },
8      adresa: { type: String, trim: true },
9      password: { type: String, required: true },
10     role: { type: String, enum: ["client", "contractor", "admin"],
11       default: "client" },
12   },
13   { timestamps: true }
14 );

```

Listing 1: Shema korisnika (models/User.js)

Polje `email` označeno je kao `unique`, čime MongoDB na razini indeksa sprječava registraciju dva korisnika s istom adresom e-pošte, dodatno uz provjeru koja

se izvodi u ruti za registraciju (potpoglavlje 4.1.2). Opcija `timestamps: true` automatski dodaje polja `createdAt` i `updatedAt` svakom dokumentu, što se koristi za sortiranje zapisa po datumu kreiranja u administratorskom pregledu.

```
1 export const KATEGORIJE = [  
2   "vlaga_i_fleke", "pukotine", "krov", "vodoinstalacije",  
3   "elektroinstalacije", "fasada", "podovi_i_zidne_obloge",  
4   "izolacija", "plijesan", "stolarija", "ostalo",  
5 ];  
6  
7 const companySchema = new mongoose.Schema(  
8   {  
9     naziv: { type: String, required: true, trim: true },  
10    telefon: { type: String, required: true, trim: true },  
11    email: { type: String, required: true, trim: true },  
12    grad: { type: String, trim: true },  
13    opis: { type: String, trim: true },  
14    kategorije: {  
15      type: [String],  
16      enum: KATEGORIJE,  
17      required: true,  
18      validate: (arr) => arr.length > 0,  
19    },  
20  },  
21  { timestamps: true }  
22 );
```

Listing 2: Shema obrtnika (models/Company.js)

Konstanta `KATEGORIJE` definira zatvoren popis mogućih kategorija građevinskih problema i izvezena je iz modela kako bi je mogli koristiti i drugi dijelovi backend sustava – prvenstveno servis za AI analizu (potpoglavlje 4.1.3), koji vraćenu kategoriju uspoređuje upravo s ovim popisom. Prilagođena funkcija u polju `validate` osigurava da svaki obrtnik ima barem jednu kategoriju, čime se sprječava zapis firme koja se ne bi mogla pojaviti ni u jednoj preporuci.

```
1 const analysisSchema = new mongoose.Schema(  
2   {  
3     user: { type: mongoose.Schema.Types.ObjectId, ref: "User",  
4       required: true },  
5     slika: { type: String, required: true },  
6     opisKorisnika: { type: String, default: "", trim: true },  
7  
8     naslovProblema: { type: String, required: true },  
9     kategorija: { type: String, required: true },  
10    opisProblema: { type: String, required: true },  
11    preporuceno_rjesenje: { type: String, required: true },  
12    hitnost: { type: String, enum: ["niska", "srednja", "visoka"],  
13      default: "srednja" },  
14    procjenaTroskaMin: { type: Number },  
15    procjenaTroskaMax: { type: Number },  
16    pouzdanost: { type: Number, min: 0, max: 1 },  
17  
18    preporuceneFirme: [{ type: mongoose.Schema.Types.ObjectId, ref:  
19      : "Company" }],  
20  },  
21 );
```

```

18   { timestamps: true }
19 );

```

Listing 3: Shema analize (models/Analysis.js)

Polje `pouzdanost` bilježi razinu sigurnosti AI modela u postavljenu dijagnozu, u rasponu od 0 do 1, i koristi se na frontendu za upozorenje korisnika kad slika nije bila dovoljno jasna (potpoglavlje 4.2). Polje `preporuceneFirme` sadrži niz referenci na kolekciju `Company`, popunjava se automatski nakon AI analize prema prepoznatoj kategoriji (potpoglavlje 4.1.3) i predstavlja implementaciju veze više-na-više opisane konceptualnim modelom (slika 2).

```

1  const interestSchema = new mongoose.Schema(
2    {
3      user: { type: mongoose.Schema.Types.ObjectId, ref: "User",
4              required: true },
5      company: { type: mongoose.Schema.Types.ObjectId, ref: "Company",
6                 required: true },
7      analysis: { type: mongoose.Schema.Types.ObjectId, ref: "
8                  Analysis", required: true },
9      status: { type: String, enum: ["na_cekaju", "kontaktirano", "
10                                zavrsono"], default: "na_cekaju" },
11    },
12    { timestamps: true }
13 );

```

Listing 4: Shema izraženog interesa (models/Interest.js)

Shema `User` definira osnovne podatke korisnika te polje `role` s ograničenim skupom vrijednosti (`client`, `contractor`, `admin`), dok se lozinka pohranjuje isključivo kao bcrypt hash, nikad u izvornom obliku. Shema `Company` sadrži polje `kategorije` – niz stringova ograničen na unaprijed definiran popis mogućih kategorija problema, koji AI analiza koristi za automatsko povezivanje s odgovarajućim obrtnicima. Shema `Analysis` referencira korisnika koji je analizu pokrenuo (`user`) putem MongoDB `ObjectId` reference, čuva rezultat AI analize (naslov, kategoriju, opis, preporučeno rješenje, hitnost te procjenu troška kao raspon minimalne i maksimalne vrijednosti) te niz referenci na preporučene obrtnike (`preporuceneFirme`). Shema `Interest` povezuje tri reference – korisnika, obrtnika i analizu – uz polje `status` koje prati tijek izraženog interesa od trenutka prijave do završetka. Referencijalni integritet, za razliku od relacijskih baza, ovdje nije nametnut na razini same baze podataka (MongoDB ne provjerava postoji li referencirani dokument), već se oslanja na ispravnost aplikacijske logike koja te reference postavlja.

3.8 Dizajn REST API-ja

API je organiziran prema REST arhitekturnom stilu – putanje predstavljaju resurse (imenice), dok HTTP metoda određuje akciju nad tim resursom. Sustav izlaže pet glavnih grupa resursa, sažetih u tablici 2.

Svaki resurs slijedi standardnu REST konvenciju: GET metoda koristi se za dohvaćanje podataka bez izmjene stanja na serveru, POST za kreiranje novih zapisa (npr. registracija korisnika, kreiranje nove analize), a DELETE za brisanje

Resurs	Opis	Zaštićen
/api/auth	Registracija i prijava korisnika	Djelomično
/api/analysis	Prijava problema i AI analiza	Da
/api/companies	Pregled i upravljanje obrtnicima	Djelomično
/api/interest	Izražavanje interesa za obrtnika	Da
/api/admin	Administratorski pregled podataka	Da (admin)

Tablica 2: Pregled REST resursa sustava

postojećih zapisa (npr. uklanjanje obrtnika iz administratorskog panela). Zaštićene rute zahtijevaju valjan JWT token u `Authorization` zaglavlju zahtjeva, koji middleware funkcija provjerava prije nego dopusti izvršavanje route handlera.

4 Implementacija

U ovom ćemo poglavlju detaljno je opisana implementacija backend i frontend dijela aplikacije, uz stvarne isječke koda, preuzete iz projekta.

4.1 Implementacija poslužiteljskog dijela

4.1.1 Struktura i pokretanje servera

Ulazna točka backend aplikacije (`index.js`) postavlja Express aplikaciju, registrira middleware lanac i sve grupe ruta, te uspostavlja vezu s bazom podataka prije nego server počne primati zahtjeve. CORS je konfiguriran, tako da eksplicitno provjerava dolazni `Origin` zahtjeva protiv popisa dopuštenih adresa iz varijable okoline `CLIENT_ORIGIN`, umjesto, da dopušta pristup bilo kojoj domeni.

```
1  const allowedOrigins = (process.env.CLIENT_ORIGIN || "http://
   localhost:5173")
2    .split(",")
3    .map((o) => o.trim());
4
5  app.use(
6    cors({
7      origin(origin, callback) {
8        if (!origin || allowedOrigins.includes(origin)) {
9          return callback(null, true);
10         }
11         callback(new Error("CORS: ova domena nije dopustena."));
12       },
13     })
14 );
15
16 app.use(express.json());
17 app.use("/uploads", uploadsRoutes);
18 app.use("/api/auth", authRoutes);
19 app.use("/api/analysis", analysisRoutes);
20 app.use("/api/companies", companyRoutes);
21 app.use("/api/interest", interestRoutes);
22 app.use("/api/admin", adminRoutes);
23
24 app.use(notFoundHandler);
25 app.use(errorHandler);
```

Listing 5: CORS konfiguracija i registracija ruta (`index.js`)

Važno je da su `notFoundHandler` i `errorHandler` registrirani *nakon* svih ruta – Express middleware funkcije izvršavaju se redoslijedom kojim su registrirane, pa bi njihovo postavljanje prije ruta spriječilo da ijedan zahtjev uopće dođe do stvarne obrade.

4.1.2 Autentikacija

Autentikacija je implementirana kroz middleware funkciju koja provjerava JWT token iz `Authorization` zaglavlja prije nego dopusti pristup zaštićenoj ruti:

```

1 export default function auth(req, res, next) {
2   const header = req.headers.authorization || "";
3   const token = header.startsWith("Bearer ") ? header.slice(7) :
      null;
4
5   if (!token) {
6     return res.status(401).json({ message: "Niste prijavljeni." });
7   }
8
9   try {
10    const decoded = jwt.verify(token, process.env.JWT_SECRET);
11    req.userId = decoded.id;
12    req.userRole = decoded.role;
13    next();
14  } catch {
15    return res.status(401).json({ message: "Nevazeci token." });
16  }
17 }

```

Listing 6: JWT middleware za provjeru autentikacije (middleware/auth.js)

Sam proces registracije hashira lozinku algoritmom bcrypt prije spremanja u bazu, dok prijava uspoređuje unesenu lozinku s pohranjenim hashom te, ako se poklapaju, generira potpisan JWT token:

```

1 function makeToken(user) {
2   return jwt.sign({ id: user._id, role: user.role }, process.env.
      JWT_SECRET, {
3     expiresIn: "7d",
4   });
5 }
6
7 router.post(
8   "/register",
9   registerValidation,
10  validate,
11  asyncHandler(async (req, res) => {
12    const { ime, prezime, email, password, telefon, adresa } = req
      .body;
13
14    const existing = await User.findOne({ email: email.toLowerCase
      () });
15    if (existing) {
16      return res.status(400).json({ message: "Email je vec
        registriran." });
17    }
18
19    const hashed = await bcrypt.hash(password, 10);
20    const user = await User.create({
21      ime, prezime, email, telefon, adresa,
22      password: hashed,
23      role: "client",
24    });
25

```

```

26     res.status(201).json({ token: makeToken(user), user:
27         publicUser(user) });
28 });

```

Listing 7: Registracija i generiranje JWT tokena (routes/auth.js)

Uloga `admin` namjerno se ne može postaviti kroz javnu registraciju – svaki novi korisnik dobiva ulogu `client`, a administratorski računi postavljaju se ručno izravno u bazi podataka, čime se sprječava da bilo tko sebi dodijeli administratorske ovlasti putem API-ja.

4.1.3 Obrada fotografije i AI analiza

Upload fotografije obrađuje se pomoću `multer` middlewarea, koji ograničava dopuštene tipove datoteka i maksimalnu veličinu:

```

1  const upload = multer({
2      storage,
3      limits: { fileSize: 8 * 1024 * 1024 }, // 8MB
4      fileFilter: (req, file, cb) => {
5          if (!file.mimetype.startsWith("image/")) {
6              return cb(new Error("Dozvoljene su samo slike."));
7          }
8          cb(null, true);
9      },
10 });

```

Listing 8: Konfiguracija `multer` middlewarea (routes/analysis.js)

Nakon uploada, sadržaj fotografije šalje se Anthropic Claude API-ju zajedno sa sistemskim uputama koje modelu definiraju točan format očekivanog odgovora (strukturiran JSON):

```

1  export async function analyzePhoto(imageBuffer, originalname,
2      description) {
3      const base64Image = imageBuffer.toString("base64");
4      const mediaType = mimeTypeFor(originalname);
5
6      const response = await anthropic.messages.create({
7          model: MODEL,
8          max_tokens: 1024,
9          system: SYSTEM_PROMPT,
10         messages: [
11             {
12                 role: "user",
13                 content: [
14                     { type: "image", source: { type: "base64", media_type:
15                         mediaType, data: base64Image } },
16                     { type: "text", text: userText },
17                 ],
18             },
19         ],
20     });

```

```

20  const textBlock = response.content.find((b) => b.type === "text
    ");
21  const parsed = extractJson(textBlock.text);
22
23  if (!KATEGORIJE.includes(parsed.kategorija)) {
24    parsed.kategorija = "ostalo";
25  }
26
27  return parsed;
28 }

```

Listing 9: Poziv Claude vision API-ja za analizu fotografije (services/aiAnalysis.js)

Odgovor modela dodatno se validira nakon parsiranja – ako vraćena kategorija ne odgovara unaprijed definiranom popisu dopuštenih kategorija, sustav je postavlja na vrijednost "ostalo" umjesto da bez provjere pohrani proizvoljan tekst koji je model mogao vratiti.

Fotografije se, za razliku od uobičajene prakse serviranja statičkih datoteka putem `express.static()`, ne serviraju javno, već prolaze kroz vlastitu autoriziranu rutu koja provjerava token i vlasništvo nad analizom prije vraćanja slike:

```

1  router.get("/:filename", async (req, res) => {
2    const token = getTokenFromRequest(req);
3    if (!token) {
4      return res.status(401).json({ message: "Niste prijavljeni." });
5    }
6
7    const decoded = jwt.verify(token, process.env.JWT_SECRET);
8    const filename = path.basename(req.params.filename); //
    sprjecava path traversal
9    const filePath = path.join(UPLOAD_DIR, filename);
10
11    if (decoded.role !== "admin") {
12      const analysis = await Analysis.findOne({ slika: `/uploads/${
        filename}` });
13      if (!analysis || analysis.user.toString() !== decoded.id) {
14        return res.status(403).json({ message: "Nemate pristup ovoj
        slici." });
15      }
16    }
17
18    res.sendFile(filePath);
19  });

```

Listing 10: Autorizirani pristup fotografijama (routes/uploads.js)

Naziv datoteke dodatno se propušta kroz `path.basename()`, čime se sprječava tzv. *path traversal* napad – pokušaj pristupa datotekama izvan predviđenog `uploads` direktorija korištenjem putanja poput `"../../index.js"`.

4.1.4 Centralizirano rukovanje greškama

Sve asinkrone rute omotane su pomoćnom funkcijom `asyncHandler`, koja hvata odbačene (rejected) Promise-e i prosljeđuje ih Express error-handling middlewa-

reu, umjesto da neuhvaćena greška sruši server:

```
1 export default function asyncHandler(fn) {
2   return function (req, res, next) {
3     Promise.resolve(fn(req, res, next)).catch(next);
4   };
5 }
```

Listing 11: asyncHandler wrapper (middleware/asyncHandler.js)

Centralizirani error handler zatim prepoznaje vrstu greške i vraća odgovarajući HTTP status kod i poruku:

```
1 export function errorHandler(err, req, res, next) {
2   if (err.name === "CastError") {
3     return res.status(400).json({ message: "Neispravan format ID-a",
4       details: err.message });
5   }
6   if (err.name === "ValidationError") {
7     return res.status(400).json({ message: "Neispravni podaci.",
8       details: err.message });
9   }
10  if (err.code === 11000) {
11    return res.status(409).json({ message: "Zapis s tim podatkom
12      vec postoji." });
13  }
14  if (err.name === "MulterError") {
15    return res.status(400).json({ message: err.message });
16  }
17  res.status(err.status || 500).json({ message: "Greska na serveru",
18    details: err.message });
19 }
```

Listing 12: Centralizirani error handler (middleware/errorHandler.js)

4.1.5 Orkestracija AI analize

Ruta koja povezuje upload fotografije, poziv AI servisa i preporuku obrtnika predstavlja središnji dio poslovne logike sustava. Za razliku od funkcije `analyzePhoto` (potpoglavlje 4.1.3), koja je zadužena isključivo za komunikaciju s Anthropic Claude API-jem, route handler u nastavku orkestrira cijeli tijek: prima datoteku, poziva AI servis, dohvaća odgovarajuće obrtnike iz baze te sprema rezultat.

```
1 router.post(
2   "/",
3   auth,
4   upload.single("photo"),
5   uploadAnalysisValidation,
6   validate,
7   asyncHandler(async (req, res) => {
8     if (!req.file) {
9       return res.status(400).json({ message: "Fotografija je
10         obavezna." });
11     }
12   })
13 )
```

```

12     let aiResult;
13     try {
14         const imageBuffer = fs.readFileSync(req.file.path);
15         aiResult = await analyzePhoto(imageBuffer, req.file.
            originalname, req.body.description);
16     } catch (err) {
17         console.error("Greska pri AI analizi:", err.message);
18         return res.status(502).json({ message: "AI servis trenutno
            nije dostupan. Pokusaj ponovno." });
19     }
20
21     let companies = await Company.find({ kategorije: aiResult.
        kategorija }).limit(3);
22     if (companies.length === 0) {
23         companies = await Company.find({ kategorije: "ostalo" }).
            limit(3);
24     }
25
26     const analysis = await Analysis.create({
27         user: req.userId,
28         slika: `/uploads/${req.file.filename}`,
29         opisKorisnika: req.body.description || "",
30         naslovProblema: aiResult.naslovProblema,
31         kategorija: aiResult.kategorija,
32         opisProblema: aiResult.opisProblema,
33         preporuceno_rjesenje: aiResult.preporuceno_rjesenje,
34         hitnost: aiResult.hitnost,
35         procjenaTroskaMin: aiResult.procjenaTroskaMin,
36         procjenaTroskaMax: aiResult.procjenaTroskaMax,
37         pouzdanost: aiResult.pouzdanost,
38         preporuceneFirme: companies.map((c) => c._id),
39     });
40
41     const populated = await analysis.populate("preporuceneFirme");
42     res.status(201).json(populated);
43 })
44 );

```

Listing 13: Ruta za prijavu problema i AI analizu (routes/analysis.js)

Poziv prema AI servisu omotan je vlastitim `try/catch` blokom, odvojenim od globalnog `asyncHandler` mehanizma (potpoglavlje 4.1.4), jer nedostupnost vanjskog servisa nije obična programska greška, već očekivano stanje kojemu treba vratiti specifičan HTTP status 502 (Bad Gateway) i razumljivu poruku, umjesto generičke poruke o grešci na serveru. Nakon uspješne AI analize, sustav dohvaća do tri obrtnika registrirana za prepoznatu kategoriju; ako takvih nema, kao rezervna opcija koriste se obrtnici iz kategorije "ostalo", čime se korisniku uvijek nudi barem okvirna preporuka. Metoda `populate()` zatim zamjenjuje reference obrtnika njihovim punim dokumentima prije slanja odgovora, tako da frontend odmah dobiva sve podatke potrebne za prikaz preporučenih obrtnika (potpoglavlje 4.2), bez dodatnog zahtjeva prema API-ju.

4.1.6 Rute za obrtnike, izražene interese i administraciju

Preostale tri grupe resursa iz tablice 2 slijede istu arhitekturu middleware lanca opisanu ranije u ovom poglavlju, uz razlike u razini potrebnih ovlasti. Autorizacija na razini role provodi se pomoćnom funkcijom `requireAdmin`, koja nadopunjuje JWT middleware iz potpoglavlja 4.1.2:

```
1 export function requireAdmin(req, res, next) {
2   if (req.userRole !== "admin") {
3     return res.status(403).json({ message: "Nemate ovlasti (samo
      admin)." });
4   }
5   next();
6 }
```

Listing 14: Provjera administratorske role (middleware/auth.js)

Resurs `/api/companies` djelomično je zaštićen: dohvaćanje popisa obrtnika javno je dostupno (koristi ga i neprijavljeni korisnik pri pregledu preporuka), dok su dodavanje i brisanje obrtnika rezervirani isključivo za administratora.

```
1 router.get(
2   "/",
3   asyncHandler(async (req, res) => {
4     const companies = await Company.find().sort({ naziv: 1 });
5     res.json(companies);
6   })
7 );
8
9 router.post(
10  "/",
11  auth,
12  requireAdmin,
13  createCompanyValidation,
14  validate,
15  asyncHandler(async (req, res) => {
16    const { naziv, telefon, email, grad, opis, kategorije } = req.
      body;
17    const company = await Company.create({ naziv, telefon, email,
      grad, opis, kategorije });
18    res.status(201).json(company);
19  })
20 );
21
22 router.delete(
23   "/:id",
24   auth,
25   requireAdmin,
26   companyIdValidation,
27   validate,
28   asyncHandler(async (req, res) => {
29     const company = await Company.findByIdAndDelete(req.params.id)
30     ;
31     if (!company) return res.status(404).json({ message: "Firma ne
      postoji." });
32     res.json({ message: "Firma obrisana." });
33   })
34 );
```

```
32   })
33   );
```

Listing 15: Rute za obrtnike (routes/companies.js)

Registracija obrtnika kao samostalnih korisnika sustava namjerno nije implementirana – za potrebe opsega ovog rada, obrtnike u sustav unosi isključivo administrator, čime se pojednostavljuje tijek bez narušavanja osnovne funkcionalnosti povezivanja korisnika s odgovarajućim izvođačima.

Resurs `/api/interest` u potpunosti je zaštićen te povezuje tri entiteta iz konceptualnog modela: korisnika, obrtnika i konkretnu analizu za koju je interes izražen.

```
1  router.post(
2    "/",
3    auth,
4    createInterestValidation,
5    validate,
6    asyncHandler(async (req, res) => {
7      const { companyId, analysisId } = req.body;
8
9      const analysis = await Analysis.findById(analysisId);
10     if (!analysis) return res.status(404).json({ message: "Analiza
11       ne postoji." });
12     if (analysis.user.toString() !== req.userId) {
13       return res.status(403).json({ message: "Ova analiza ne
14         pripada vama." });
15     }
16
17     const interest = await Interest.create({
18       user: req.userId,
19       company: companyId,
20       analysis: analysisId,
21     });
22
23     res.status(201).json(interest);
24   })
25 );
26
27 router.get(
28   "/",
29   auth,
30   asyncHandler(async (req, res) => {
31     const interests = await Interest.find({ user: req.userId })
32       .populate("company")
33       .populate("analysis")
34       .sort({ createdAt: -1 });
35     res.json(interests);
36   })
37 );
```

Listing 16: Rute za izražene interese (routes/interest.js)

Prije kreiranja zapisa interesa, sustav provjerava pripada li navedena analiza upravo prijavljenom korisniku – ista autorizacijska logika koja se koristi i pri pris-

tupu fotografijama (potpoglavlje 4.1.3), čime se sprječava da korisnik izrazi interes u ime tuđe analize izravnim pozivom prema API-ju.

Resurs `/api/admin`, konačno, u cijelosti je rezerviran za administratorsku ulogu. Umjesto ponavljanja `auth` i `requireAdmin` middlewarea na svakoj pojedinoj ruti, oba se primjenjuju jednom, na razini cijelog routera:

```
1 router.use(auth, requireAdmin);
2
3 router.get(
4   "/users",
5   asyncHandler(async (req, res) => {
6     const users = await User.find().select("-password").sort({
7       createdAt: -1 });
8     res.json(users);
9   })
10 );
11
12 router.get(
13   "/analyses",
14   asyncHandler(async (req, res) => {
15     const analyses = await Analysis.find()
16       .populate("user", "ime prezime email")
17       .populate("preporuceneFirme")
18       .sort({ createdAt: -1 });
19     res.json(analyses);
20   })
21 );
22
23 router.get(
24   "/interests",
25   asyncHandler(async (req, res) => {
26     const interests = await Interest.find()
27       .populate("user", "ime prezime email")
28       .populate("company")
29       .populate("analysis")
30       .sort({ createdAt: -1 });
31     res.json(interests);
32   })
33 );
```

Listing 17: Rute za administratorski pregled (routes/admin.js)

Poziv `router.use(auth, requireAdmin)` na početku datoteke primjer je iskorištavanja Express middleware lanca za organizaciju zajedničke odgovornosti (opisano u potpoglavlju 2.2) – svaki zahtjev prema bilo kojoj ruti unutar ovog routera prolazi kroz obje provjere prije nego dođe do odgovarajućeg handlera, bez potrebe da se to ponavlja za svaku rutu zasebno. Metoda `.select("-password")` pritom eksplicitno isključuje polje lozinke (i kad je riječ o hashu, a ne izvornoj vrijednosti) iz odgovora prema administratorskom sučelju, kao dodatna mjera opreza uz sam bcrypt hash opisan u potpoglavlju 2.4.

4.2 Implementacija korisničkog dijela

U ovom ćemo potpoglavlju opisana je implementacija korisničkog dijela aplikacije: zajednička Axios instanca za komunikaciju s backendom, router guard za ograničavanje pristupa stranicama, komponenta za prijavu problema i prikaz AI dijagnoze, te administratorski panel.

4.2.1 Axios instanca i interceptor

Sva komunikacija s backendom odvija se kroz jednu zajedničku axios instancu s unaprijed postavljenom baznom adresom. Request interceptor automatski dodaje JWT token u zaglavlje svakog odlaznog zahtjeva, čime se izbjegava ručno dodavanje tokena u svakoj komponenti zasebno:

```
1  const api = axios.create({
2    baseUrl: "/api",
3  });
4
5  api.interceptors.request.use((config) => {
6    const token = localStorage.getItem("token");
7    if (token) {
8      config.headers.Authorization = `Bearer ${token}`;
9    }
10   return config;
11 });
12
13 export function imageUrl(path) {
14   if (!path) return "";
15   const token = localStorage.getItem("token");
16   return `${path}${token ? `?token=${encodeURIComponent(token)}` : ""}`;
17 }
```

Listing 18: Axios instanca s interceptorom (src/api.js)

Funkcija `imageUrl()` rješava specifičan problem: HTML `<img>` element ne može slati prilagođena (custom) zaglavlja, poput `Authorization` zaglavlja koje interceptor dodaje ostalim zahtjevima. Budući da su fotografije zaštićene autoriziranom rutom (potpoglavlje 4.1.3), token se za prikaz slika prosljeđuje kao query parametar unutar same adrese slike.

4.2.2 Router guard

Pristup pojedinim stranicama aplikacije ograničen je prema meta poljima definiranim za svaku rutu (`requiresAuth`, `guestOnly`, `requiresAdmin`), koje globalni navigacijski guard, provjerava prije svake promjene rute:

```
1  const routes = [
2    { path: "/", name: "Auth", component: Auth, meta: { guestOnly: true } },
3    { path: "/pocetna", name: "Home", component: Home, meta: { requiresAuth: true } },
4    { path: "/admin", name: "Admin", component: Admin, meta: { requiresAuth: true, requiresAdmin: true } },
```

```

5 ];
6
7 router.beforeEach((to) => {
8   const token = localStorage.getItem("token");
9   const role = localStorage.getItem("role");
10
11   if (to.meta.requiresAuth && !token) {
12     return { name: "Auth" };
13   }
14   if (to.meta.guestOnly && token) {
15     return { name: "Home" };
16   }
17   if (to.meta.requiresAdmin && role !== "admin") {
18     return { name: "Home" };
19   }
20 });

```

Listing 19: Router guard (src/router/index.js)

Slika 4 prikazuje sučelje za prijavu (komponenta **Auth.vue**), na koje router guard preusmjerava svakog neautenticiranog korisnika koji pokuša pristupiti zaštićenoj ruti, te na koje se korisnik vraća i nakon odjave. Nakon uspješnog poziva prema `/api/auth/login`, JWT token vraćen s backenda pohranjuje se u `localStorage`, odakle ga axios interceptor (Listing 18) automatski dodaje u zaglavlje svakog sljedećeg zahtjeva, dok meta polje `guestOnly` sprječava povratak na ovaj ekran dok je korisnik već prijavljen.

Slika 4: Sučelje za prijavu (Auth.vue)

Analogno tome, slika 5 prikazuje formu za registraciju unutar iste komponente. Njezina polja (ime, prezime, opcionalni telefon i adresa, email, lozinka) izravno odgovaraju parametrima koje backend ruta `/api/auth/register` očekuje u tijelu zahtjeva (Listing 7); svaki novi korisnik pritom dobiva ulogu `client` bez obzira na unesene podatke, u skladu s ograničenjem opisanim u potpoglavlju 4.1.2.

TataMata

Uslikaj problem, dobij stručnu dijagnozu i preporuku majstora

Registracija

Ime

Prezime

Telefon (opcionalno)

Adresa (opcionalno)

Email

Lozinka

Registriraj se

Slika 5: Sučelje za registraciju (Auth.vue)

4.2.3 Komponenta za prijavu problema i prikaz dijagnoze

Glavna funkcionalnost aplikacije – upload fotografije, prikaz AI dijagnoze i popis preporučenih obrtnika – implementirana je unutar jedne Vue komponente (`Home.vue`), organizirane oko tri reaktivna stanja: forme za unos, rezultata analize i povijesti prethodnih prijava. Slika 6 prikazuje ovu komponentu u prijavljenom stanju (gumb `Odjava` u zaglavlju, prisutan zahvaljujući meta polju `requiresAuth` iz Listinga 19): gornji dio sadrži formu za upload fotografije i unos opisa problema, dok donji dio prikazuje popis prethodno podnesenih prijava (`Prijašnje prijave`), dohvaćen zasebnim pozivom prema `/api/analysis` pri učitavanju komponente.

TataMata Odjava

Prijavi problem

Uslikaj problem u kući ili stanu (npr. fleku na zidu, pukotinu, oštećenje) i po potrebi dodaj kratak opis. AI će ti reći o čemu se vjerojatno radi i predložiti će rješenje.

Fotografija

Odaberi datoteku
datoteka nije odabrana

Opis (opcionalno)

npr. velika mrlja na zidu, pojavila se nakon kiše

Analiziraj

Prijašnje prijave

Probijeni nosivi/pregradni zid za novi prolaz
03. 09. 2026.

ostalo

Slika 6: Forma za prijavu problema i povijest prijašnjih prijava (Home.vue)

```

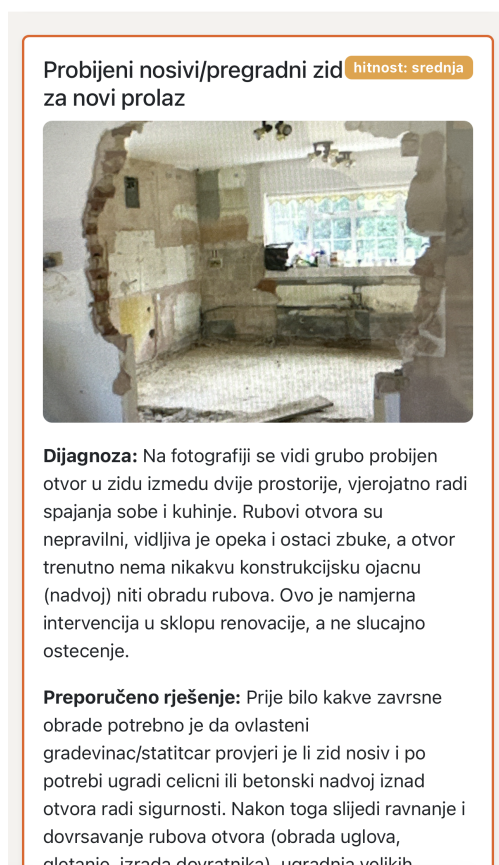
1  async function submitAnalysis() {
2    uploadError.value = "";
3    const file = fileInput.value.files[0];
4    if (!file) {
5      uploadError.value = "Odaberi fotografiju.";
6      return;
7    }
8
9    analyzing.value = true;
10   result.value = null;
11   interestSentFor.value = new Set();
12
13   try {
14     const formData = new FormData();
15     formData.append("photo", file);
16     formData.append("description", description.value);
17
18     const { data } = await api.post("/analysis", formData, {
19       headers: { "Content-Type": "multipart/form-data" },
20     });
21
22     result.value = data;
23     await loadHistory();
24   } catch (err) {
25     uploadError.value = err.response?.data?.message || "Greška pri
       analizi. Pokušaj ponovno.";
26   } finally {
27     analyzing.value = false;
28   }
29 }

```

Listing 20: Slanje fotografije na analizu (src/views/Home.vue)

Funkcija koristi `FormData` objekt umjesto običnog JSON tijela zahtjeva, jer se uz tekstualni opis šalje i binarni sadržaj fotografije – `multipart/form-data` format koji na backendu prima `multer` middleware (potpoglavlje 4.1.3). Stanje `analyzing` koristi se za onemogućavanje ponovnog slanja forme i prikaz poruke o učitavanju dok traje poziv prema AI servisu, čime se zadovoljava nefunkcionalni zahtjev odzivnosti sučelja opisan u potpoglavlju 3.3.

Nakon uspješnog odgovora, korisniku se prikazuje puna dijagnoza s mogućnošću izražavanja interesa za svaku pojedinu preporučenu firmu. Slika 7 prikazuje rezultat pohranjen u reaktivnoj varijabli `result` nakon poziva iz Listinga 20: naslov problema s oznakom hitnosti, učitanu fotografiju, te tekstualni dio dijagnoze i početak preporučenog rješenja koje je vratio Anthropic Claude API (potpoglavlje 4.1.3).



Slika 7: Prikaz AI dijagnoze – naslov, hitnost, fotografija i opis problema (Home.vue)

```
1 async function expressInterest(companyId) {  
2   try {  
3     await api.post("/interest", { companyId, analysisId: result.  
        value._id });
```

```

4     interestSentFor.value = new Set([...interestSentFor.value,
      companyId]);
5   } catch (err) {
6     alert(err.response?.data?.message || "Greška.");
7   }
8 }

```

Listing 21: Izražavanje interesa za obrtnika (src/views/Home.vue)

Skup `interestSentFor` pamti identifikatore obrtnika za koje je interes već poslan unutar tekuće sesije pregleda rezultata, čime se u predlošku komponente onemogućuje ponovno slanje zahtjeva za istog obrtnika (gumb mijenja natpis u "Poslano" i postaje neaktivan) bez potrebe za dodatnim upitom prema backendu. Na razini backenda, poziv prema `/api/interest` (Listing 16) prije kreiranja zapisa provjerava pripada li navedena analiza upravo prijavljenom korisniku, nakon čega se u kolekciji `Interest` stvara novi zapis koji povezuje korisnika, obrtnika i konkretnu analizu sa statusom `"na_cekaju"`. Aplikacija u ovoj fazi ne šalje automatsku obavijest obrtniku – zapis interesa služi kao evidencija namjere koju administrator naknadno pregledava u kartici `Interesi` administratorskog panela (potpoglavlje 4.2.4) te po potrebi ručno kontaktira obje strane; automatizacija te obavijesti (npr. slanjem e-pošte obrtniku) ostaje kao mogućnost budućeg proširenja. U predlošku komponente dodatno se korisniku ističe napomena kada je AI pouzdanost dijagnoze niska:

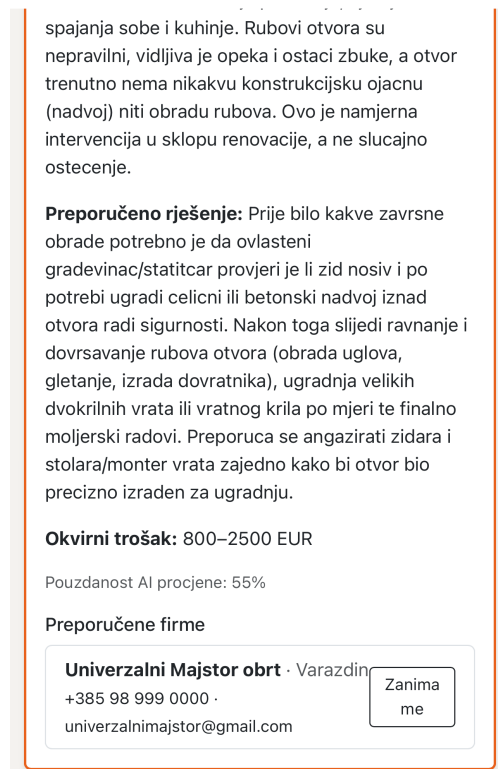
```

1 <p class="text-muted small mb-3">
2   Pouzdanost AI procjene: {{ Math.round(result.pouzdanost * 100)
      }}%
3   <span v-if="result.pouzdanost < 0.5">-- slika nije bila potpuno
      jasna, preporuka je okvirna.</span>
4 </p>

```

Listing 22: Prikaz upozorenja pri niskoj pouzdanosti dijagnoze (src/views/Home.vue)

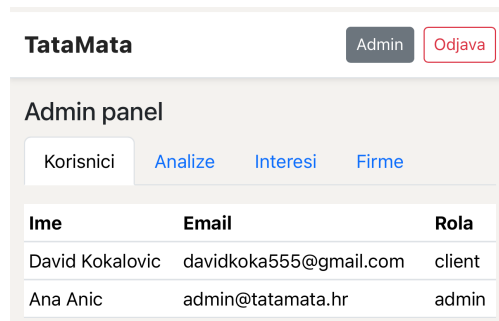
Polje `pouzdanost`, definirano u shemi analize (Listing 3), na ovaj se način izravno koristi za upravljanje korisničkim očekivanjima – umjesto da se dijagnoza niske pouzdanosti prikaže identično kao i svaka druga, korisnik se eksplicitno upozorava da rezultat treba uzeti s rezervom. Slika 8 prikazuje nastavak istog prikaza rezultata: kraj preporučenog rješenja, procjenu troška, postotak pouzdanosti AI procjene te odjeljak `Preporučene firme` s gumbom `Zanima me` iz Listinga 21, koji ostaje aktivan sve dok korisnik ne izrazi interes za tog konkretnog obrtnika.



Slika 8: Procjena troška, pouzdanost AI procjene i preporučene firme s mogućnošću izražavanja interesa (Home.vue)

4.2.4 Administratorski panel

Administratorsko sučelje (`Admin.vue`) organizirano je kroz četiri kartice koje odgovaraju administratorskim resursima iz tablice 2: pregled korisnika, analiza, izraženih interesa te upravljanje obrtnicima. Pristup cijelom panelu ograničen je na ulogu `admin` kroz meta polje `requiresAdmin` (Listing 19) – gumb `Admin` u zaglavlju vidljiv je svim prijavljenim korisnicima, no router guard preusmjerava korisnika s ulogom `client` natrag na početnu stranicu i onemogućuje izravan pristup ruti. Slika 9 prikazuje karticu `Korisnici`: tablični prikaz svih registriranih korisnika s njihovom ulogom, dobiven pozivom prema `/api/admin/users` (Listing 17), pri čemu je polje lozinke isključeno iz odgovora metodom `.select("-password")`. Svi podaci za preostale tri kartice dohvaćaju se paralelno, pri učitavanju komponente:



Slika 9: Administratorski panel – kartica Korisnici (Admin.vue)

```

1  async function loadAll() {
2    const [u, a, i, c] = await Promise.all([
3      api.get("/admin/users"),
4      api.get("/admin/analyses"),
5      api.get("/admin/interests"),
6      api.get("/companies"),
7    ]);
8    users.value = u.data;
9    analyses.value = a.data;
10   interests.value = i.data;
11   companies.value = c.data;
12 }

```

Listing 23: Paralelno dohvaćanje administratorskih podataka (src/views/Admin.vue)

Korištenje `Promise.all()` umjesto četiri uzastopna `await` poziva omogućuje da se svi zahtjevi izvrše istovremeno, čime se ukupno vrijeme čekanja svodi na trajanje najsporijeg pojedinog poziva, umjesto zbroja trajanja svih četiriju. Kartica za upravljanje obrtnicima (slika 10) dodatno sadrži formu za dodavanje novog obrtnika te gumb **Obriši** uz svaki postojeći zapis:

Admin panel

Korisnici
Analize
Interesi
Firme

Naziv
Grad

Telefon
Email

Br. stavki: 0

Drži Ctrl (Cmd na Macu) za odabir više kategorija.

Dodaj firmu

Naziv	Kategorije	
AquaServis obrt	vodoinstalacije, vlaga_i_fleke	Obriši
CistiZid obrt za plijesan	plijesan, vlaga_i_fleke	Obriši
Elektro Maras	elektroinstalacije	Obriši
Fasada Plus	fasada, izolacija	Obriši
FixIt Sve d.o.o.	pukotine, podovi_i_zidne_obloge	Obriši
ProBuild d.o.o.	vlaga_i_fleke, krov, izolacija	Obriši
Univerzalni Majstor obrt	ostalo, stolarija	Obriši

Slika 10: Administratorski panel – kartica Firme, s formom za dodavanje i popisom postojećih obrtnika (Admin.vue)

```

1  async function addCompany() {
2    companyError.value = "";
3    companySuccess.value = "";
4    try {
5      await api.post("/companies", newCompany.value);
6      companySuccess.value = "Firma dodana.";
7      newCompany.value = { naziv: "", grad: "", telefon: "", email:
8        "", kategorije: [] };
9      await loadAll();
10   } catch (err) {
11     companyError.value = err.response?.data?.message || "Greška.";
12   }
13 }
14
15 async function deleteCompany(id) {
16   if (!confirm("Obrisati firmu?")) return;
17   await api.delete(`/companies/${id}`);
18   await loadAll();
19 }

```

Listing 24: Dodavanje i brisanje obrtnika (src/views/Admin.vue)

Nakon uspješnog dodavanja ili brisanja obrtnika, komponenta ponovno poziva `loadAll()` kako bi prikazani popis ostao usklađen sa stanjem baze podataka, umjesto da se novi zapis ručno umeće u lokalno stanje – jednostavniji pristup prik-

ladan za opseg ove aplikacije, premda uz cijenu dodatnog mrežnog zahtjeva pri svakoj izmjeni. Brisanje obrtnika dodatno je zaštićeno potvrdnim dijalogom preglednika (`confirm()`), kao zadnja linija obrane od slučajnog klika, iako je stvarna autorizacijska provjera već provedena na backendu putem `requireAdmin` middlewarea (Listing 14).

4.3 Sigurnosni aspekti implementacije

Sigurnost aplikacije oslanja se na nekoliko neovisnih mehanizama koji zajedno pokrivaju različite razine sustava. Lozinke se nikad ne pohranjuju niti prenose u izvornom obliku – hashiraju se berypt algoritmom prije spremanja u bazu, a pri prijavi se ponovno hashiraju i uspoređuju s pohranjenim hashom, umjesto obrnutog dešifriranja. JWT token nosi identitet i ulogu korisnika te se provjerava na svakoj zaštićenoj ruti neovisno o frontend provjerama, čime se ostvaruje stateless autentikacija u skladu s REST arhitekturnim principima.

Validacija ulaznih podataka provodi se dvoslojno: HTML5 atributi na frontendu (npr. `required, type="email"`) daju brzu povratnu informaciju korisniku, dok `express-validator` lanci na backendu predstavljaju stvarnu sigurnosnu granicu, jer se frontend provjere mogu zaobići izravnim pozivom prema API-ju. CORS konfiguracija ograničava koje su domene uopće ovlaštene pozivati API, a autorizirani pristup fotografijama sprječava neovlašteno gledanje tuđih uploada putem izravnog pogađanja putanje datoteke. Zajedno, ovi mehanizmi pokrivaju autentikaciju, autorizaciju, integritet podataka i zaštitu osjetljivog sadržaja (fotografija) unutar sustava.

4.4 Proširenja sustava

Uz osnovnu funkcionalnost opisanu u prethodnim potpoglavljima, sustav je naknadno proširen s dvanaest dodatnih funkcionalnosti, koje unaprjeđuju korisničko iskustvo (samostalna registracija obrtnika, ocjenjivanje, uređivanje profila), administratorski uvid u sustav (statistika, pretraga, filtriranje, sortiranje, izvoz podataka) te ukupnu robusnost sustava (brisanje analiza, upravljanje statusom interesa).

4.4.1 Samostalna registracija obrtnika

Model `User` od početka je predviđao ulogu `contractor` u polju `role`, no ta se uloga inicijalno mogla dodijeliti isključivo ručno, putem administratorskog sučelja. Registracijska forma proširena je poljem za odabir uloge obrtnika, čijim odabirom korisnik uz standardne podatke unosi i podatke o vlastitom obrtu (naziv, kontakt, kategorije rada). Backend ruta `POST /api/auth/register` pritom, unutar iste transakcijske cjeline, kreira i zapis korisnika i pripadajući zapis obrtnika u kolekciji `Company`, povezan poljem `vlasnik`. Sučelje za registraciju obrtnika prikazano je na slici 11.

21:43  5G 50%

☒ Registriram se kao obrtnik (dodajem svoju firmu)

Podaci o obrtu

Naziv obrta

Zid.d.o.o.o.

Telefon obrta

0987654321

Grad

Zagreb

Kategorije rada

Stavke: 4

Drži Ctrl (Cmd na Macu) za odabir više kategorija.

Email

zid@gmail.com

Lozinka

••••••••

Registriraj se

[Već imaš račun?](#)  Powered by Netlify

tatamata-zavrsni-frontend.netlify.app

Slika 11: Sučelje za samostalnu registraciju obrtnika (Auth.vue)

4.4.2 Ocjenjivanje obrtnika

Model `Interest` proširen je poljima `ocjena` (cjelobrojna vrijednost 1–5) i `komentar`. Nakon izraženog interesa, korisniku se prikazuje forma za ocjenjivanje obrtnika, čije slanje poziva rutu `PATCH /api/interest/:id/ocjena`:

```
1 router.patch(
```

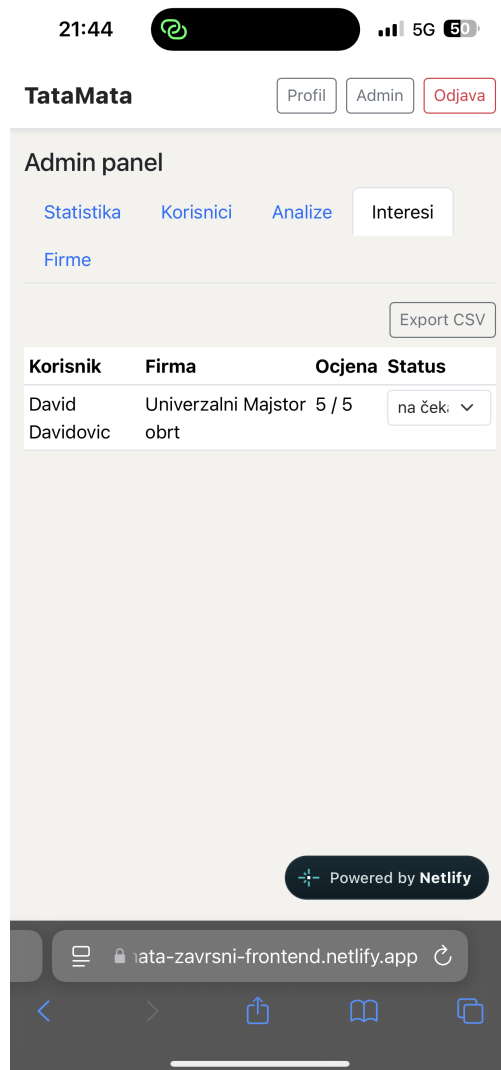
```

2  "/:id/ocjena",
3  auth,
4  rateInterestValidation,
5  validate,
6  asyncHandler(async (req, res) => {
7    const interest = await Interest.findById(req.params.id);
8    if (!interest)
9      return res.status(404).json({ message: "Interes ne postoji."
10      });
11    if (interest.user.toString() !== req.userId) {
12      return res.status(403).json({ message: "Ovaj interes ne
13      pripada vama." });
14    }
15    interest.ocjena = req.body.ocjena;
16    interest.komentar = req.body.komentar || "";
17    await interest.save();
18    res.json(interest);
19  });

```

Listing 25: Ruta za ocjenjivanje obrtnika (routes/interest.js)

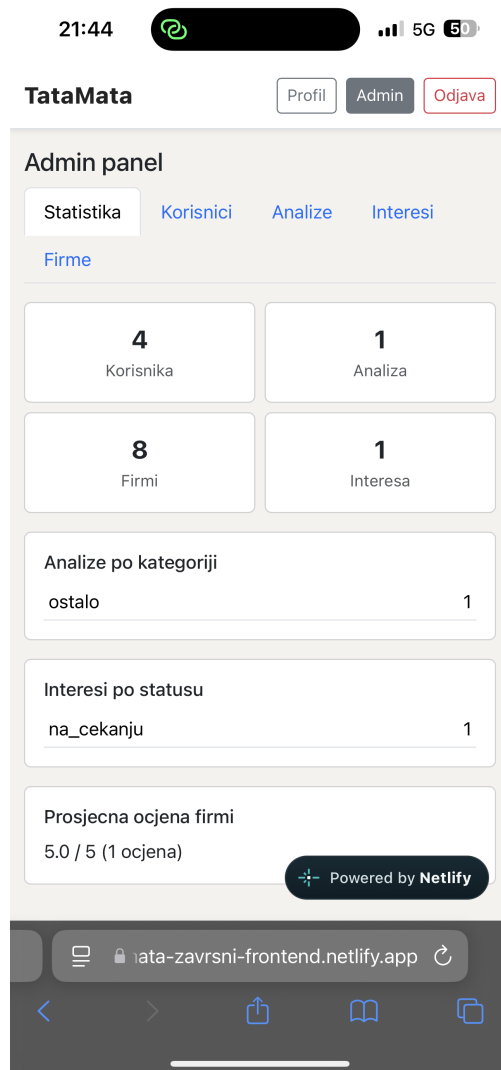
Provjera vlasništva nad zapisom (`interest.user.toString() !== req.userId`) osigurava da korisnik može ocijeniti isključivo vlastiti izraženi interes, u skladu s nefunkcionalnim zahtjevom autorizacije na razini resursa (potpoglavlje 3.3). Administratorski uvid u unesene ocjene, zajedno sa statusom pojedinog interesa (potpoglavlje 4.4.5), prikazan je na slici 12.



Slika 12: Administratorski panel – kartica Interesi, s prikazom ocjene i statusa (Admin.vue)

4.4.3 Statistika sustava

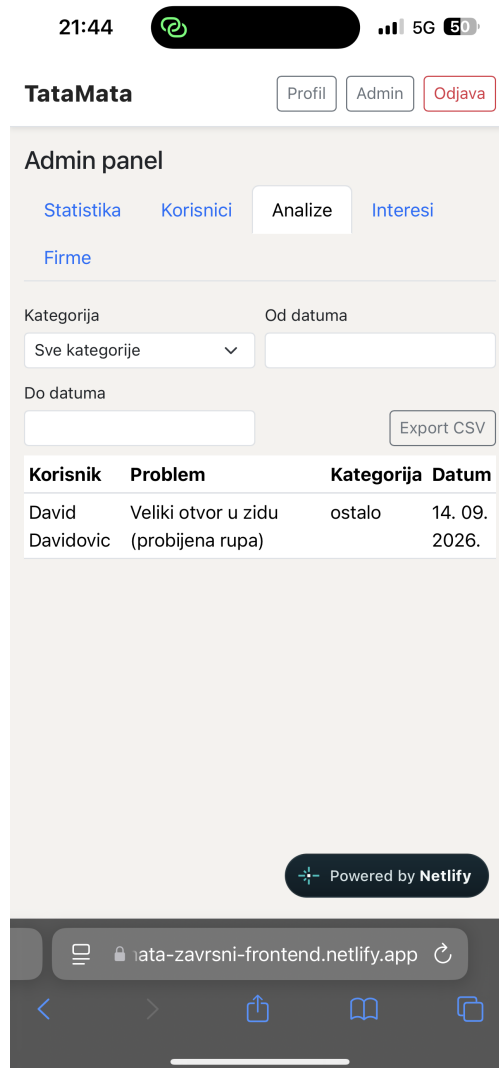
Ruta GET `/api/admin/stats` koristi MongoDB agregacijski cjevovod (*aggregation pipeline*) za izračun objedinjenih pokazatelja – ukupan broj korisnika, analiza, obrtnika i interesa, broj analiza grupiran po kategoriji problema, broj interesa grupiran po statusu, te prosječnu ocjenu svih obrtnika. Rezultat se prikazuje u novom, prvom tabu administratorskog sučelja (slika 13).



Slika 13: Administratorski panel – kartica Statistika (Admin.vue)

4.4.4 Pretraga, filtriranje, sortiranje i izvoz podataka

Preostale nadogradnje administratorskog sučelja odvijaju se u potpunosti na strani klijenta, nad podacima već dohvaćenima pri učitavanju sučelja, bez dodatnih poziva prema backendu: pretraga obrtnika po nazivu ili gradu, filtriranje analiza po kategoriji i rasponu datuma, sortiranje svake tablice klikom na zaglavlje stupca, te izvoz prikazanih podataka u CSV format putem Blob sučelja preglednika. Ovakav pristup smanjuje opterećenje backend sustava i ubrzava odziv sučelja, u skladu s nefunkcionalnim zahtjevom odzivnosti (potpoglavlje 3.3), budući da navedene operacije ne ovise o mrežnoj latenciji. Slika 14 prikazuje karticu **Analize** s poljima za filtriranje po kategoriji i rasponu datuma, te gumbom za izvoz filtriranog prikaza u CSV.



Slika 14: Administratorski panel – kartica Analize, s filtrom po kategoriji i datumu (Admin.vue)

Uz pretragu na frontendu, ruta `GET /api/companies` dodatno koristi MongoDB agregacijski cjevovod s operacijom `$lookup` kako bi za svaku firmu izbrojala povezane zapise u kolekciji `Interest` i dodala izračunato polje `brojInteresa`, prikazano kao dodatni stupac u admin tablici firmi (slika 15) – korisno za brzi uvid u to koje su firme trenutno najtraženije.

```

1 router.get(
2   "/",
3   asyncHandler(async (req, res) => {
4     const companies = await Company.aggregate([
5       { $sort: { naziv: 1 } },
6       {
7         $lookup: {
8           from: "interests",
9           localField: "_id",
10          foreignField: "company",
11          as: "interesi",

```

```

12     },
13     },
14     { $addField: { brojInteresa: { $size: "$interesi" } } },
15     { $project: { interesi: 0 } },
16   });
17   res.json(companies);
18 }
19 );

```

Listing 26: Brojanje izraženih interesa po firmi (routes/companies.js)

21:44 5G 50

Naziv Grad

Telefon Email

Br. stavki: 0

Drži Ctrl (Cmd na Macu) za odabir više kategorija.

Dodaj firmu

Pretraži firme po nazivu ili gradu... Export CSV

Naziv	Kategorije	Interesa	
AquaServis obrt	vodoinstalacije, vlaga_i_fleke	0	Obriši
CistiZid obrt za plijesan	plijesan, vlaga_i_fleke	0	Obriši
Elektro Maras	elektroinstalacije	0	Obriši
Fasada Plus	fasada, izolacija	0	Obriši
MarinKrov	krov	0	Obriši
ProBuild d.o.o.	vlaga_i_fleke, krov, izolacija	0	Obriši
Univerzalni Majstor obrt	ostalo, stolarija	1	Obriši
Zid.d.o.o.o.	pukotine, fasada, izolacija, plijesan	0	Obriši

Powered by Netlify

ata-zavrsni-frontend.netlify.app

Slika 15: Administratorski panel – kartica Firme, s pretragom i brojem interesa po firmi (Admin.vue)

4.4.5 Upravljanje statusom interesa i brisanje analiza

Administratorsko sučelje omogućuje promjenu statusa izraženog interesa (na čekanju, kontaktirano, završeno) putem rute PATCH /api/interest/:id, zaštićene

requireAdmin middlewareom, prikazane u istoj kartici Interesi kao i na slici 12 (potpoglavlje 4.4.2):

```
1 router.patch(  
2   "("/:id",  
3   auth,  
4   requireAdmin,  
5   asyncHandler(async (req, res) => {  
6     const { status } = req.body;  
7     const DOZVOLJENI_STATUSEI = ["na_cekaju", "kontaktirano", "  
8       zavrsono"];  
9     if (!DOZVOLJENI_STATUSEI.includes(status)) {  
10       return res.status(400).json({ message: "Neispravan status."  
11     });  
12   }  
13   const interest = await Interest.findByIdAndUpdate(  
14     req.params.id,  
15     { status },  
16     { new: true, runValidators: true }  
17   );  
18   if (!interest) return res.status(404).json({ message: "Interes  
19     ne postoji." });  
20   res.json(interest);  
21 })  
22 );
```

Listing 27: Promjena statusa izraženog interesa (routes/interest.js)

Korisnicima je omogućeno i brisanje vlastite analize (DELETE /api/analysis/:id), pri čemu ruta, uz provjeru vlasništva, briše pripadajuću fotografiju s diska te sve povezane zapise u kolekciji Interest, čime se sprječava nastanak nekonzistentnih zapisa u bazi podataka:

```
1 router.delete(  
2   "("/:id",  
3   auth,  
4   analysisIdValidation,  
5   validate,  
6   asyncHandler(async (req, res) => {  
7     const analysis = await Analysis.findById(req.params.id);  
8     if (!analysis) return res.status(404).json({ message: "Analiza  
9       ne postoji." });  
10    if (analysis.user.toString() !== req.userId) {  
11      return res.status(403).json({ message: "Nemate pravo  
12        obrisati ovu analizu." });  
13    }  
14  
15    const filePath = path.join(UPLOAD_DIR, path.basename(analysis.  
16      slika));  
17    fs.unlink(filePath, () => {});  
18  
19    await Interest.deleteMany({ analysis: analysis._id });  
20    await analysis.deleteOne();  
21  
22    res.json({ message: "Analiza obrisana." });  
23  })  
24 );
```

21);

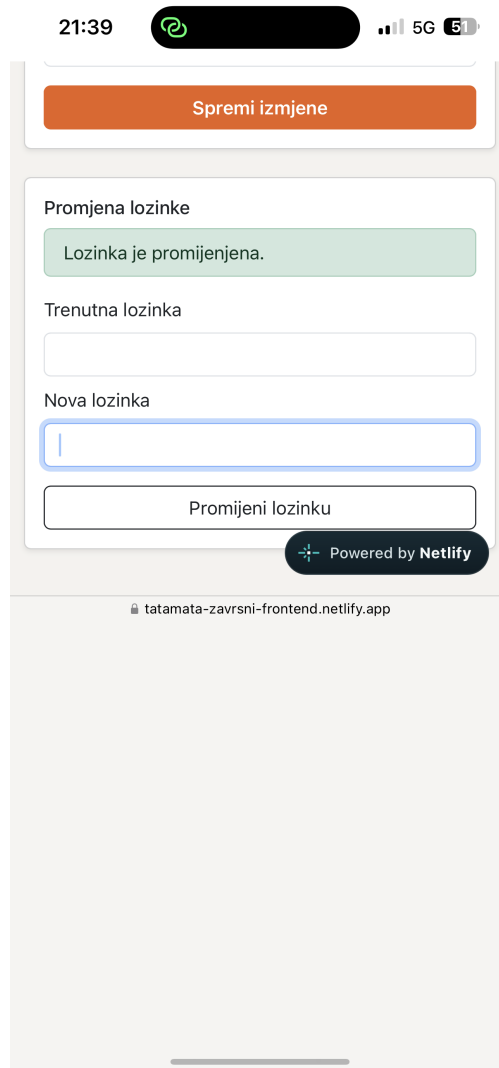
Listing 28: Brisanje analize i povezanih zapisa (routes/analysis.js)

4.4.6 Uređivanje profila i promjena lozinke

Nova stranica /profil (Profile.vue) korisniku omogućuje uređivanje osobnih podataka i promjenu lozinke kroz dvije odvojene forme. Prva šalje zahtjev prema PATCH /api/auth/me (ime, prezime, telefon, adresa). Druga, sigurnosno osjetljivija, šalje zahtjev prema PATCH /api/auth/me/lozinka te zahtijeva unos trenutne lozinke, koja se uspoređuje s pohranjenim hashom prije nego što se dopusti postavljanje nove – čime se sprječava da netko tko privremeno dođe do otključanog uređaja korisnika promijeni lozinku bez poznavanja stvarne.

```
1 router.patch(  
2   "/me/lozinka",  
3   auth,  
4   changePasswordValidation,  
5   validate,  
6   asyncHandler(async (req, res) => {  
7     const { trenutnaLozinka, novaLozinka } = req.body;  
8     const user = await User.findById(req.userId);  
9     if (!user) return res.status(404).json({ message: "Korisnik ne  
10      postoji." });  
11  
12     const match = await bcrypt.compare(trenutnaLozinka, user.  
13       password);  
14     if (!match) {  
15       return res.status(400).json({ message: "Trenutna lozinka  
16         nije ispravna." });  
17     }  
18  
19     user.password = await bcrypt.hash(novaLozinka, 10);  
20     await user.save();  
21     res.json({ message: "Lozinka je uspješno promijenjena." });  
22   })  
23 );
```

Listing 29: Promjena lozinke uz provjeru trenutne (routes/auth.js)



Slika 16: Stranica Moj profil – forma za promjenu lozinke (Profile.vue)

4.4.7 Toast obavijesti

Preglednikov `alert()` poziv, korišten u ranijoj verziji sučelja, blokira cijelu stranicu dok korisnik ručno ne potvrdi poruku. Zamijenjen je lakom, vlastitom komponentom temeljenom na Vue `reactive()` funkciji, koja omogućuje dijeljeno stanje dostupno bilo kojoj komponenti bez potrebe za dodatnom bibliotekom za upravljanje stanjem (npr. Pinia):

```

1 export const toasts = reactive([]);
2
3 let nextId = 1;
4
5 function push(message, type) {
6   const id = nextId++;
7   toasts.push({ id, message, type });
8   setTimeout(() => remove(id), 4000);
9 }
10

```

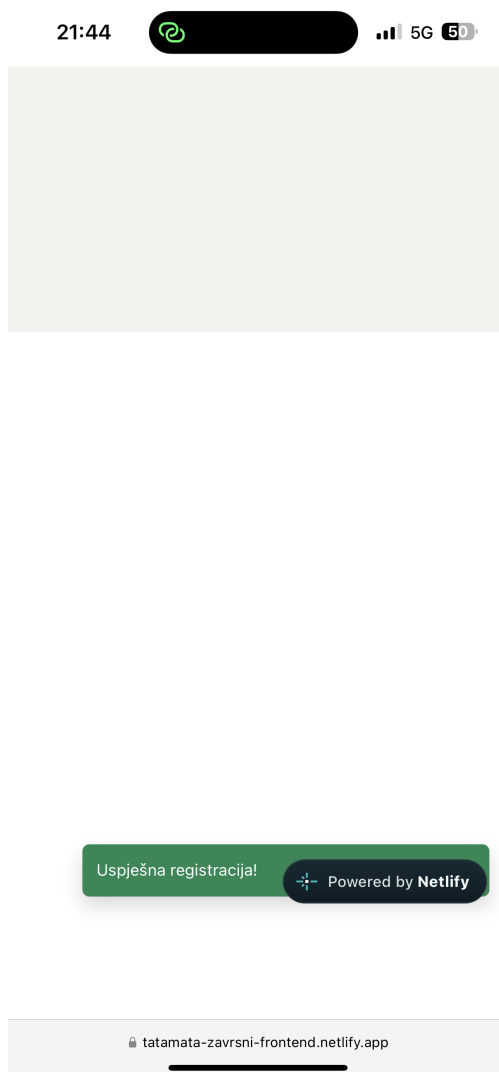
```

11 export function showSuccess(message) {
12     push(message, "success");
13 }
14
15 export function showError(message) {
16     push(message, "error");
17 }

```

Listing 30: Globalni spremnik toast poruka (src/toast.js)

Komponenta `Toast.vue` iscertava trenutno aktivne poruke u donjem desnom kutu zaslona, obojane prema tipu (zelena za uspjeh, crvena za grešku), a svaka poruka automatski se uklanja nakon 4 sekunde putem `setTimeout()` poziva. Funkcije `showSuccess()` i `showError()` pozivaju se iz komponenti `Auth.vue`, `Admin.vue` i `Home.vue`, čime je jedinstven način obavještanja korisnika dosljedno primijenjen kroz cijelu aplikaciju.



Slika 17: Toast obavijest o uspješnoj registraciji

5 Testiranje

Testiranje aplikacije provedeno je kroz niz User Acceptance Testova (UAT) – ručnih scenarija kojima se tijekom razvoja provjeravalo ponaša li se sustav u skladu s definiranim funkcionalnim i nefunkcionalnim zahtjevima (potpoglavlja 3.2 i 3.3) iz perspektive krajnjeg korisnika. Svaki scenarij opisuje korak koji korisnik ili administrator izvodi te očekivani ishod sustava. Provedeni scenariji sažeti su u nastavku:

- **Registracija i prijava korisnika.** Testirana je registracija novog korisnika i prijava postojećeg, uz unos neispravnih podataka (već zauzet email, prekratka lozinka). Sustav u svim slučajevima vraća odgovarajuću poruku greške i odbija zahtjev. *Rezultat: uspješno.*
- **AI analiza fotografije.** Testiran je upload različitih fotografija građevinskih problema (vlaga, pukotine, oštećenje krova). AI analiza u svakom slučaju vraća smislen i strukturiran odgovor – kategoriju, hitnost i procjenu troška. *Rezultat: uspješno.*
- **Nejasna fotografija.** Testiran je upload fotografije koja ne prikazuje jasan građevinski problem. Sustav vraća dijagnozu s niskom vrijednošću polja **pouzdanost**, u skladu sa sistemskim uputama AI modela. *Rezultat: uspješno.*
- **Pristup bez autentikacije.** Testiran je pristup zaštićenim rutama bez valjanog JWT tokena. Sustav u svakom slučaju vraća status kod 401 (Unauthorized). *Rezultat: uspješno.*
- **Pristup bez administratorskih ovlasti.** Testiran je pristup administratorskim rutama s računom koji nema ulogu **admin**. Sustav vraća status kod 403 (Forbidden). *Rezultat: uspješno.*
- **Pristup tuđoj fotografiji.** Testiran je pristup fotografiji koja pripada tuđoj analizi, izravnim pogađanjem putanje. Sustav vraća status kod 403, čime je potvrđena autorizacijska logika opisana u potpoglavlju 4.1.3. *Rezultat: uspješno.*
- **CORS zaštita.** Testiran je poziv prema API-ju s domene koja nije na popisu dopuštenih CORS adresa. Kako middleware takvu grešku prosljeđuje dalje bez definiranog statusnog koda, **errorHandler** je obrađuje generičkim statusom 500 uz poruku "Greška na serveru." *Rezultat: uspješno.*
- **Nedostupnost AI servisa.** Testirana je situacija u kojoj Anthropic Claude API privremeno nije dostupan tijekom analize fotografije. Sustav vraća status kod 502 (Bad Gateway) s porukom "AI servis trenutno nije dostupan. Pokušaj ponovno.", u skladu s obradom greške opisanom u potpoglavlju 4.1.5, umjesto da neuhvaćena greška sruši server. Ovaj scenarij izravno provjerava nefunkcionalni zahtjev robusnosti na pogreške vanjskih servisa definiran u potpoglavlju 3.3 – tijekom razvoja aplikacija je stvarno naišla

na ovu situaciju, čime je scenarij potvrđen i u praksi, a ne samo namjerno izazvan. *Rezultat: uspješno.*

- **Ocjenjivanje tuđeg interesa.** Testiran je pokušaj ocjenjivanja izraženog interesa koji pripada drugom korisniku, izravnim pogađanjem ID-a zapisa. Sustav vraća status kod 403, čime je potvrđena ista autorizacijska logika primijenjena i na ostale zaštićene resurse. *Rezultat: uspješno.*
- **Samostalna registracija obrtnika.** Testirana je registracija korisnika s označenom ulogom obrtnika, uz unos podataka o obrtu. Sustav uspješno kreira i korisnički račun i pripadajući zapis obrtnika, koji se odmah pojavljuje u popisu preporučenih obrtnika za odabrane kategorije. *Rezultat: uspješno.*
- **Brisanje analize.** Testirano je brisanje vlastite analize korisnika, uz provjeru da se povezani izraženi interesi također uklanjaju iz baze podataka, a pokušaj brisanja tuđe analize odbija se statusom 403. *Rezultat: uspješno.*

Uz ručno provedene User Acceptance Testove, dodan je i skup jediničnih testova (Vitest) koji pokrivaju ključne dijelove poslovne logike neovisne o vanjskim servisima: centralizirano rukovanje greškama (`errorHandler`), autentikacijski middleware (`auth`, `requireAdmin`), te validacijska pravila Mongoose sheme obrtnika (`Company`). Svi testovi izvršavaju se bez potrebe za stvarnom bazom podataka ili mrežnim pozivima, čime se mogu pokrenuti brzo i ponovljivo, primjerice u sklopu CI procesa. S obzirom na opseg aplikacije, ovakav pristup User Acceptance Testiranja bio je dovoljan za provjeru osnovne funkcionalnosti tijekom razvoja. Za produkcijsku verziju aplikacije ili daljnji razvoj, sljedeći bi korak bio uvođenje automatiziranih testova – jediničnih testova (npr. Vitest, Jest) za funkcije neovisne o vanjskim servisima te integracijskih testova za API rute (provjera statusnih kodova i strukture odgovora za svaki resurs).

6 Zaključak

U okviru ovog završnog rada kreirana je web aplikacija¹ koja korisnicima omogućuje automatiziranu dijagnostiku građevinskih problema na temelju fotografije i tekstualnog opisa, uz preporuku lokalnih obrtnika, prilagođenih prepoznatoj kategoriji problema. Sustav je izgrađen kao dva neovisna dijela koji komuniciraju putem REST API-ja – Node.js/Express.js backend s MongoDB Atlas bazom podataka i Vue.js 3 frontend – uz integraciju Anthropic Claude API-ja za analizu fotografija. U radu su opisane korištene tehnologije, arhitektura i model podataka sustava, te implementacija ključnih funkcionalnosti: autentikacije putem JWT tokena, obrade uploada fotografija, poziva vanjskog AI servisa i sigurnosnih mehanizama koji štite korisničke podatke.

Osim osnovne funkcionalnosti, sustav je naknadno proširen dvanaestom dodatnim funkcionalnostima opisanim u potpoglavlju 4.4: samostalnom registracijom obrtnika, ocjenjivanjem obrtnika, uređivanjem korisničkog pretragom, filtriranjem, sortiranjem i izvozom podataka u administratorskom sučelju. Aplikacija je javno objavljena – backend na usluzi Render, a frontend na usluzi Netlify – čime je ostvaren i potpuno funkcionalan, javno dostupan prototip, neovisan o lokalnom razvojnom okruženju.

Cilj rada – razvoj funkcionalne aplikacije koja spaja modernu web arhitekturu s vanjskim AI servisom radi automatizacije zadatka koji je tradicionalno zahtijevao stručnu procjenu na terenu – je u potpunosti ostvaren, uz navedena proširenja koja nadilaze inicijalno postavljene zahtjeve.

Aplikacija u trenutnom obliku i dalje ima određena ograničenja: postojeći automatizirani testovi ograničeni su na osnovne jedinične testove te ne pokrivaju cijeli sustav (nedostaju integracijski testovi API ruta), a popisi podataka (obrtnici, analize) prikazuju se u cijelosti bez straniciranja, što bi pri znatno većem broju zapisa moglo utjecati na odzivnost sučelja. Dodatno, korištena besplatna razina usluge Render uzrokuje privremenu obustavu (*spin down*) poslužitelja nakon perioda neaktivnosti, zbog čega prvi sljedeći zahtjev može biti odgođen i do minute – ograničenje svojstveno odabranoj besplatnoj infrastrukturi, a ne implementaciji sustava. Moguća, buduća proširenja aplikacije uključuju: uvođenje straniciranja za veće količine podataka, proširenje testnog pokrića integracijskim testovima za API rute, uvođenje globalnog upravljanja stanjem (npr. Pinia) ukoliko aplikacija naraste u opsegu, te izravnu komunikaciju (chat) između korisnika i obrtnika unutar same aplikacije, umjesto oslanjanja isključivo na kontakt podatke prikazane uz profil obrtnika.

¹Potpuni izvorni kod dostupan je na GitHub repozitorijima: <https://github.com/dkokalovi/tatamata-zavrsni-backend>
<https://github.com/dkokalovi/tatamata-zavrsni-frontend>

Literatura

- [1] Vue.js. Vue.js – the progressive javascript framework. <https://vuejs.org/guide/introduction.html>, 2024. Pristupljeno: 2. rujna 2026.
- [2] Vite. Vite documentation. <https://vitejs.dev/guide/>, 2024. Pristupljeno: 2. rujna 2026.
- [3] Vue Router. Vue router – the official router for vue.js. <https://router.vuejs.org/>, 2024. Pristupljeno: 2. rujna 2026.
- [4] Axios. Axios docs. <https://axios-http.com/docs/intro>, 2024. Pristupljeno: 2. rujna 2026.
- [5] OpenJS Foundation. Express – node.js web application framework. <https://expressjs.com/>, 2024. Pristupljeno: 2. rujna 2026.
- [6] MongoDB, Inc. Mongoddb manual. <https://www.mongodb.com/docs/manual/>, 2024. Pristupljeno: 2. rujna 2026.
- [7] Kristina Chodorow and Michael Dirolf. *MongoDB: The Definitive Guide*. O'Reilly Media, 2010.
- [8] Mongoose. Mongoose odm v8.x – guide. <https://mongoosejs.com/docs/guide.html>, 2024. Pristupljeno: 2. rujna 2026.
- [9] M. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). RFC 7519, IETF, 2015.
- [10] OWASP Foundation. Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2024. Pristupljeno: 2. rujna 2026.
- [11] Anthropic. Claude api documentation. <https://docs.anthropic.com/en/api/getting-started>, 2024. Pristupljeno: 2. rujna 2026.

Popis slika

1	Arhitektura sustava	7
2	Konceptualni model podataka	9
3	Implementacija Mongoose shema	10
4	Sučelje za prijavu (Auth.vue)	24
5	Sučelje za registraciju (Auth.vue)	25
6	Forma za prijavu problema i povijest prijašnjih prijava (Home.vue)	26
7	Prikaz AI dijagnoze – naslov, hitnost, fotografija i opis problema (Home.vue)	27
8	Procjena troška, pouzdanost AI procjene i preporučene firme s mo- gućnošću izražavanja interesa (Home.vue)	29
9	Administratorski panel – kartica Korisnici (Admin.vue)	30
10	Administratorski panel – kartica Firme, s formom za dodavanje i popisom postojećih obrtnika (Admin.vue)	31
11	Sučelje za samostalnu registraciju obrtnika (Auth.vue)	33
12	Administratorski panel – kartica Interesi, s prikazom ocjene i sta- tusa (Admin.vue)	35
13	Administratorski panel – kartica Statistika (Admin.vue)	36
14	Administratorski panel – kartica Analize, s filtrom po kategoriji i datumu (Admin.vue)	37
15	Administratorski panel – kartica Firme, s pretragom i brojem inte- resa po firmi (Admin.vue)	38
16	Stranica Moj profil – forma za promjenu lozinke (Profile.vue)	41
17	Toast obavijest o uspješnoj registraciji	42

Popis tablica

1	Pregled aktera i slučajeva korištenja sustava	8
2	Pregled REST resursa sustava	13